

The Ultimate JSBSim Reference

Architecture, Aerodynamics, and Aircraft Modelling — Under the Hood

A practical engineering manual covering JSBSim's XML configuration language, equations of motion, integration methods, aerodynamic function/table framework, propulsion subsystem, flight control components, and the workflow for generating CFD-derived aerodynamic data tables for a custom aircraft model.

Generated 2026-05-24 · Reverse-engineered from JSBSim v2.0 source

Table of Contents

Chapter 1: Introduction to JSBSim	6
1.1 What this document covers	6
1.2 How JSBSim differs from a typical sim engine	6
1.3 Conventions used in this document	6
Chapter 2: Architecture and Execution Flow	7
2.1 The model execution order	7
2.2 The Run loop in pseudocode	7
2.3 Modules at a glance	8
Chapter 3: Coordinate Frames, Axes, and Units	9
3.1 The structural frame (XML geometry)	9
3.2 The body-fixed (body) frame	9
3.3 Wind (aerodynamic) and stability frames	9
3.4 Local NED and ECI frames	9
3.5 Sign conventions	9
3.6 Units	10
Chapter 4: The Aircraft XML, Section by Section	11
4.1 Root: fdm_config	11
4.2 fileheader: authorship and provenance	11
4.3 metrics: aircraft geometry	11
4.4 mass_balance: weight, CG, inertia	12
4.5 ground_reactions: gear, skids and contacts	13
4.6 external_reactions: bolt-on forces	13
4.7 propulsion: engines, tanks, thrusters	14
4.8 flight_control: command shaping	14
4.9 autopilot and system: same toolbox, different scope	15
4.10 aerodynamics: the core of the model	15
4.11 input and output	16
Chapter 5: Aerodynamics — Deep Dive	17
5.1 The six axes and the function/table model	17
5.2 How a single function evaluates	17
5.3 The function language	17
5.4 Tables: 1-D, 2-D, 3-D, and higher	18
5.5 Properties commonly used in aerodynamics	19
5.6 Non-dimensionalisation by hand	19
5.7 Worked example: rolling moment due to roll rate	20
5.8 Static derivatives — getting them from CFD	20

5.9 Dynamic (damping) derivatives — running CFD with motion	20
5.10 Control derivatives	21
5.11 Ground effect and Reynolds effects	21
5.12 Stall and stall hysteresis	22
Chapter 6: The Equations of Motion Under the Hood	23
6.1 State vector	23
6.2 Quaternion attitude kinematics	23
6.3 Translational dynamics in the inertial frame	23
6.4 Rotational dynamics	23
6.5 Total force and moment assembly	24
6.6 Ground friction as a constrained problem	24
6.7 Trim	24
6.8 Atmospheric model	24
6.9 Winds and turbulence	24
6.10 Property tree	25
Chapter 7: Building Your Own Aircraft, Step-by-Step	26
7.1 Step 1: Lay out the directory	26
7.2 Step 2: Skeleton XML	26
7.3 Step 3: Geometry, mass, and gear	26
7.4 Step 4: Propulsion	27
7.5 Step 5: Flight control system	27
7.6 Step 6: Aerodynamics from CFD	27
7.7 Step 7: Initial conditions and a smoke-test script	28
7.8 Step 8: Iteration loop	28
Chapter 8: CFD-to-JSBSim Workflow	29
8.1 Choosing a CFD solver	29
8.2 Recommended case matrix	29
8.3 Forced-oscillation method for damping derivatives	29
8.4 Quasi-steady method (faster)	30
8.5 Extracting and tabulating	30
8.6 Python helper: CFD CSV → JSBSim XML	30
8.7 Validation: from CFD numbers to flight feel	31
Chapter 9: Reference Appendices	32
9.1 Property tree cheat sheet	32
9.2 Common FCS components	32
9.3 Engine type cross-reference	33
9.4 Source-code map	33
9.5 Bibliography and references	33

9.6 Closing thoughts	34
Chapter 10: Visual Diagrams of the FDM Loop	35
10.1 Top-level simulation loop	35
10.2 Per-tick data flow between models	36
10.3 Body-frame, wind-frame and structural-frame sketch	36
10.4 Aerodynamic axis-to-force flow	37
Chapter 11: Propulsion — Per-Engine-Type Reference	38
11.1 Piston engines	38
11.2 Turbine (turbojet/turbofan)	38
11.3 Turboprop	39
11.4 Rocket engines	39
11.5 Electric and brushless DC motors	40
11.6 Helicopter rotor	40
11.7 Propellers	40
11.8 Fuel tanks	41
Chapter 12: Flight Control Components — Reference	42
12.1 Summer / Pure gain / Aerosurface scale	42
12.2 Filters	42
12.3 Integrator	43
12.4 PID controller	43
12.5 Kinematic, deadband, switch	43
12.6 Sensor and actuator wrapping	43
Chapter 13: Atmosphere, Winds, and Earth	45
13.1 ISA 1976 standard atmosphere	45
13.2 Winds and shear	45
13.3 Gusts (discrete)	45
13.4 Microburst	45
13.5 Inertial: gravity and Earth rotation	46
Chapter 14: Quaternion Attitude — A Primer for the Code	47
14.1 Definition and conventions	47
14.2 Kinematic equation	47
14.3 Renormalisation	47
14.4 Euler angle extraction	47
Chapter 15: Complete Worked Example: A Simple Aircraft	48
15.1 Header and reference data	48
15.2 Mass, balance and inertia	48
15.3 Landing gear	49
15.4 Propulsion	50

15.5 Flight control system	51
15.6 Aerodynamics (representative values from CFD)	52
Chapter 16: Initial Conditions, Scripts, and Resets	56
16.1 Initial conditions (reset files)	56
16.2 Scripts (event-based scenarios)	56
Chapter 17: Validation Workflow	58
17.1 Step 1: trim diagnostics	58
17.2 Step 2: open-loop step responses	58
17.3 Step 3: linear model extraction	58
17.4 Step 4: performance envelope check	58
17.5 Step 5: handling-qualities check	58
17.6 Step 6: flight-test comparison	59
Chapter 18: Common Bugs and How to Diagnose Them	60
Chapter 19: Python API	61
19.1 Quick start	61
19.2 Reading and writing properties	61
19.3 Trim and linearisation	61
19.4 Batch parameter studies	61
Chapter 20: Extended Property Reference	62
20.1 Time	62
20.2 Atmosphere and winds	62
20.3 Position	62
20.4 Attitude	62
20.5 Velocities	62
20.6 Aerodynamic state	63
20.7 Forces and moments (body frame)	63
20.8 Accelerations	63
20.9 FCS commands and positions	63
20.10 Propulsion	64
20.11 Simulation control	64
Chapter 21: Glossary and Symbol Index	65

Chapter 1: Introduction to JSBSim

JSBSim is an open-source, multi-platform, object-oriented C++ Flight Dynamics Model (FDM) used as the physics core behind a wide spectrum of aerospace applications: FlightGear, Unreal Engine's Antoinette Project, ArduPilot and PX4 Software-in-the-Loop, reinforcement-learning gyms such as *gym-jsbsim*, and academic research running into thousands of citations. It implements a nonlinear, six-degree-of-freedom rigid-body simulation with an accurate Earth model (WGS-84 ellipsoid, rotation, Coriolis), the ISA 1976 standard atmosphere, configurable wind and turbulence, propulsion (piston, turbine, turboprop, rocket, electric, rotor), ground reactions and a fully scriptable XML-based flight control system.

1.1 What this document covers

This reference is organised as a top-down dive into JSBSim. We start with the architecture and execution loop, then walk every section of the aircraft XML, then go deep on aerodynamics (the math framework, tables, and conventions), and finish with a practical CFD-to-JSBSim workflow for building your own aircraft model. Every claim is rooted in source-code citations of the form `file:line`.

1.2 How JSBSim differs from a typical sim engine

- **XML-only modelling:** aircraft are described in declarative XML — no recompilation is needed to add a new airframe.
- **Function/table aerodynamics:** aerodynamic coefficients are expressed as *functions* that compose tables, properties and arithmetic operators. This makes the model fully data-driven and easy to populate from CFD or wind-tunnel data.
- **Property tree:** every state variable, control input, FCS output and aero coefficient is exposed under a hierarchical string path (`aero/qbar-psf`, `velocities/p-aero-rad_sec`, etc.). Everything is observable and most things are writable.
- **Round-Earth physics:** equations of motion are integrated in the ECI frame, including the Coriolis and centrifugal accelerations from Earth's rotation. Optional gravitational-torque modelling supports spacecraft.
- **Trim algorithm:** a robust bracketed root-finder trims the vehicle into steady flight (longitudinal, full, turn, pull-up, ground, custom).

1.3 Conventions used in this document

Distances and inertia use US customary units by default (feet, inches, slug·ft²) — JSBSim's default — but all elements accept an explicit `unit` attribute. Math symbols follow Etkin/Stevens-Lewis convention: α = angle of attack, β = sideslip, p, q, r = body angular rates, $\bar{q} = \frac{1}{2}\rho V^2$ = dynamic pressure, $S, b, \bar{c}$ = wing area, span, mean aerodynamic chord.

Chapter 2: Architecture and Execution Flow

JSBSim is structured around a central executive class (FGFDMExec) that owns a fixed-order list of *models* and drives them through a discrete-time loop. Each model reads its inputs from the property tree (filled by upstream models in the same tick) and writes its outputs back. Integrating the equations of motion is itself one of those models.

2.1 The model execution order

Models are enumerated in FGFDMExec.h:225-241 and executed in this order every tick:

```
enum eModels {
    ePropagate=0,          // Integrate state from previous tick's accels
    eInput,                // Telnet/socket inputs
    eInertial,              // Gravity and Earth rotation vector
    eAtmosphere,            // T, p, rho, speed of sound at h
    eWinds,                 // Steady wind + gusts + turbulence
    eSystems,               // FCS, autopilot, custom systems
    eMassBalance,           // CG, inertia tensor, total mass
    eAuxiliary,             // alpha, beta, qbar, Vt, Mach, ...
    ePropulsion,            // Engine thrust and torque
    eAerodynamics,         // Aero forces and moments
    eGroundReactions,       // Gear forces and friction
    eExternalReactions,     // User-defined external forces
    eBuoyantForces,         // Lighter-than-air models
    eAircraft,              // Sum forces and moments at CG
    eAccelerations,         // Solve F = m a, tau = I omega-dot
    eOutput                 // CSV, sockets, FlightGear, ...
};
```

Why this order matters. *Propagate* runs first because it advances the state using the accelerations computed at the end of the *previous* tick. Then atmosphere and winds give pressure, density and the airmass-relative velocity that *Auxiliary* needs to compute α , β , $\bar{q}$, M — which the aerodynamics functions consume. Mass balance runs after FCS so a control law can request a pointmass shift (think: fuel transfer, stores release). Accelerations runs last; its outputs become the inputs to Propagate on the next tick.

2.2 The Run loop in pseudocode

```
FGFDMExec::Run() {                                     // FGFDMExec.cpp:404
    for child in childFDMs: child->Run()
    sim_time += dt; Frame++
    if (script) script->RunScript()
    for i in eModels:
        LoadInputs(i)          // copy upstream outputs to this model's inputs
        Models[i]->Run(holding)
}
```

Default integration step is $1.0 / 120.0$ s (see FGFDMExec.cpp:99). At 120 Hz JSBSim runs faster than real time on a single CPU core even for complex aircraft. The step is set with `SetDeltaT()` or via the simulation script.

2.3 Modules at a glance

Module	Source	Responsibility
FGPropagate	models/FGPropagate.*	Integrate position, velocity, quaternion
FGAccelerations	models/FGAccelerations.*	Newton-Euler equations; ground friction LCP
FGAerodynamics	models/FGAerodynamics.*	Sum function-driven aero forces/moments
FGPropulsion	models/FGPropulsion.*	Engines (piston/turbine/rocket/electric)
FGGroundReactions	models/FGGroundReactions.*	Tires, struts, brakes, steering
FGAtmosphere	models/FGAtmosphere.*	ISA 1976 (T, p, ρ , a)
FGWinds	models/atmosphere/FGWinds.*	Steady wind, shear, Dryden/Karman turbulence
FGInertial	models/FGInertial.*	Gravity (spherical or WGS-84), planet rotation
FGMassBalance	models/FGMassBalance.*	Mass, CG, inertia tensor, pointmasses
FGAuxiliary	models/FGAuxiliary.*	α , β , $\bar{q}$, Vt, Mach, gamma, load factors
FGFCS	models/FGFCS.*	Flight control system; channel/component graph
FGOutput	models/FGOutput.*	CSV, FlightGear binary, TCP, telnet
FGTrim	initialization/FGTrim.*	Bracketed root-finder for trim
FGPropertyManager	input_output/FGPropertyManager.*	Hierarchical property tree, all I/O via paths

Chapter 3: Coordinate Frames, Axes, and Units

Getting the frames right is the single most common source of bugs when building a new aircraft. JSBSim uses several frames at once and each section of the XML implicitly chooses one of them.

3.1 The structural frame (XML geometry)

All `<location>` elements — landing gear, CG, AERORP, pointmasses, engine thruster location — are expressed in the *structural* reference frame. This is an aircraft-fixed frame with the X axis usually pointing aft along the fuselage, Y to the right, Z up. The origin (0,0,0) is conventionally near the firewall or the nose datum. Inches are the default unit.

Inside the FDM these positions are converted to the body-fixed frame used by the equations of motion (X forward, Y right, Z down) — but you do **not** see that conversion in the XML.

3.2 The body-fixed (body) frame

All angular rates, accelerations, body-frame velocities and inertia components are in the body frame:

- **X** — forward, out through the nose.
- **Y** — out through the right wing.
- **Z** — downward, completing the right-handed triad.

Body-frame translational velocity is the property triple `velocities/u-fps`, `v-fps`, `w-fps`. Angular rates are `velocities/p-rad_sec`, `q-rad_sec`, `r-rad_sec`. The inertia tensor is given in the structural frame in the XML but is rotated to the body frame at load.

3.3 Wind (aerodynamic) and stability frames

The *wind* frame is rotated from the body frame by sideslip β and angle of attack α : its X axis points along the relative wind, drag acts in $-X$, lift acts in $-Z$ (up relative to airflow). When you write `<axis name="LIFT">` in the aerodynamics section JSBSim assumes you are in wind axes. The *stability* frame is an intermediate frame rotated from body only by α (no β); some classical stability derivatives are tabulated in that frame and you can opt into it with `frame="STABILITY"`.

JSBSim handles the conversion automatically: forces computed in *LIFT/DRAG/SIDE* are rotated by T_{w2b} into the body frame and moments are added at the AERORP, then transferred to the CG by $r \times F$.

3.4 Local NED and ECI frames

Local NED (North-East-Down) is the local-tangent frame centered at the aircraft. JSBSim uses it for winds, ground track (γ , ψ_{gt}), and the local Euler angles ϕ , θ , ψ . **ECI** (Earth-Centred Inertial) is the non-rotating frame in which the equations of motion are actually integrated; FGPropagate maintains $v_{inertial}$ and $r_{inertial}$ internally and transforms them to body/local for output.

3.5 Sign conventions

- **Elevator deflection**: positive trailing-edge-down (creates negative pitching moment), output to `fcs/elevator-pos-rad`.
- **Aileron deflection**: differential — the C-172 uses `left-aileron-pos-rad` with positive = down (rolls right).
- **Rudder deflection**: positive trailing-edge-left (yaws nose left).
- **Inertia cross products**: classical convention is I_{xz} as the integral of $+xz \, dm$. Some references use the negated convention; set `negated_crossproduct_inertia="true"` on the `<mass_balance>` element to

switch.

• **Up vs. down:** Z_{body} is positive down. The structural Z given in the XML is usually positive up — JSBSim does the flip internally.

3.6 Units

Length	FT (default), IN (default for <location>), M, KM, CM
Area	FT2 (default), M2, IN2, CM2
Mass	LBS (default), KG, SLUG (1 slug = 14.594 kg)
Inertia	SLUG*FT2 (default), KG*M2
Force	LBS, N
Pressure	PSF (default), PSI, INHG, ATM, PA, N/M2
Velocity (output)	FPS, KTS, M/S (per-property)
Angle	RAD (aero default), DEG (FCS default)
Spring	LBS/FT, N/M
Damping	LBS/FT/SEC, N/M/SEC

Always declare units explicitly on every element that accepts a `unit` attribute — silent default mismatches are responsible for the majority of "my aircraft launches itself sideways" bug reports.

Chapter 4: The Aircraft XML, Section by Section

An aircraft is a single XML file with the root element `<fdm_config>`. The order of sections is enforced by the XSD; the canonical sequence is shown below. Below the cover material follow ~10 sub-chapters, one per major section.

4.1 Root: `fdm_config`

```
<?xml version="1.0"?>
<fdm_config name="MyAircraft" version="2.0" release="PRODUCTION"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="http://jsbsim.sourceforge.net/JSBSim.xsd">

  <fileheader>          ... </fileheader>
  <planet>              ... </planet>      <!-- optional -->
  <metrics>             ... </metrics>
  <mass_balance>        ... </mass_balance>
  <ground_reactions>    ... </ground_reactions>
  <external_reactions>  ... </external_reactions>
  <buoyant_forces>      ... </buoyant_forces>
  <propulsion>          ... </propulsion>
  <system .../>         ... <!-- 0..N -->
  <autopilot ...>       ... </autopilot>    <!-- 0..1 -->
  <flight_control ...>  ... </flight_control>
  <aerodynamics>       ... </aerodynamics>
  <input ...>          ... </input>        <!-- 0..N -->
  <output ...>         ... </output>       <!-- 0..N -->
</fdm_config>
```

Attributes: name (required) — display name; version (required, currently 2.0); release = PRODUCTION | ALPHA | BETA (ALPHA/BETA print a warning at load time).

4.2 fileheader: authorship and provenance

Pure metadata. JSBSim ignores everything inside but tools that scan a fleet of models read it. Use it.

```
<fileheader>
  <author>Ada Lovelace</author>
  <email>ada@example.com</email>
  <organization>Acme Aircraft</organization>
  <license licenseName="GPL"
    licenseURL="http://www.gnu.org/licenses/gpl.html"/>
  <filecreationdate>2026-05-24</filecreationdate>
  <version>$Revision: 1.0 $</version>
  <description>Acme A-1 light sport aircraft.</description>
  <note>Stall hysteresis modelled per NACA TN-3909.</note>
  <limitation>Gear aero drag not modelled.</limitation>
  <reference title="AGARD-AR-265"
    author="ESDU"
    date="1988"
    URL="https://example.org/AR265.pdf"/>
</fileheader>
```

4.3 metrics: aircraft geometry

`<metrics>` establishes the non-dimensionalisation: S (wing area), b (wing span), $\bar{c}$ (mean aerodynamic chord), tail areas/arms, and the three *reference points* AERORP, EYEPOINT and VRP.

```
<metrics>
  <wingarea unit="FT2"> 174.0 </wingarea>
  <wingspan unit="FT" > 35.8 </wingspan>
  <chord unit="FT" > 4.9 </chord>
```

```

<htailarea unit="FT2"> 21.9 </htailarea>
<htailarm unit="FT" > 15.7 </htailarm>
<vtailarea unit="FT2"> 16.5 </vtailarea>
<vtailarm unit="FT" > 0 </vtailarm>
<wing_incidence unit="DEG"> 1.5 </wing_incidence> <!-- optional -->

<location name="AERORP" unit="IN"> <!-- 25% MAC -->
  <x>43.2</x> <y>0</y> <z>59.4</z>
</location>
<location name="EYEPOINT" unit="IN">
  <x>37</x> <y>0</y> <z>48</z>
</location>
<location name="VRP" unit="IN">
  <x>42.6</x> <y>0</y> <z>38.5</z>
</location>
</metrics>

```

The three reference points have specific meaning:

- **AERORP** — aerodynamic reference point. All aerodynamic moments are first summed about this point, then transferred to the CG by $M_{cg} = M_{arp} + r_{arp \rightarrow cg} \times F$. Conventionally placed at the 25% mean aerodynamic chord on the wing.
- **EYEPOINT** — pilot eye position. Used by FlightGear and visual systems; affects the load factor at the pilot's head (accelerations/n-pilot-{x, y, z}-norm).
- **VRP** — visual reference point, origin used by external viewers.

4.4 mass_balance: weight, CG, inertia

The empty weight plus a list of *pointmasses* is what JSBSim uses to compute total mass, CG location, and the inertia tensor at the CG. Pointmasses include crew, passengers, cargo, and user-defined items that may move at runtime (slung loads, transferable fuel). Tank mass is added automatically by the propulsion subsystem and consumed during flight.

```

<mass_balance negated_crossproduct_inertia="false">
  <ixx unit="SLUG*FT2"> 948 </ixx>
  <iyy unit="SLUG*FT2"> 1346 </iyy>
  <izz unit="SLUG*FT2"> 1967 </izz>
  <ixy unit="SLUG*FT2"> 0 </ixy>
  <ixz unit="SLUG*FT2"> 0 </ixz>
  <iyz unit="SLUG*FT2"> 0 </iyz>
  <emptywt unit="LBS"> 1500 </emptywt>
  <location name="CG" unit="IN">
    <x>41</x> <y>0</y> <z>36.5</z>
  </location>
  <pointmass name="Pilot">
    <weight unit="LBS"> 180 </weight>
    <location unit="IN"> <x>36</x><y>-14</y><z>24</z> </location>
  </pointmass>
  <pointmass name="Baggage">
    <weight unit="LBS"> 0 </weight>
    <location unit="IN"> <x>95</x><y>0</y><z>24</z> </location>
    <form shape="cylinder"> <!-- optional inertia shape -->
      <radius unit="IN"> 6 </radius>
      <length unit="IN"> 24 </length>
    </form>
  </pointmass>
</mass_balance>

```

Inertia about the CG: the values you put in *ixx*, *iyy*, *izz*, *ixy*, *ixz*, *iyz* are the inertia of the empty-weight structure about the CG, in the *structural* frame. JSBSim rotates them to the body frame, then adds the parallel-axis contributions of each pointmass. The *negated_crossproduct_inertia* attribute exists because half the textbooks use the opposite sign convention for products of inertia.

4.5 ground_reactions: gear, skids and contacts

<contact> elements describe anything that can touch the ground. There are two types:

- **BOGEY** — a wheel. Has a strut (spring + damper), tire friction (static, dynamic, rolling), optional steering, optional brakes, and optional retraction.
- **STRUCTURE** — a non-rolling contact such as a wing tip, skid, fuselage or tail bumper. Typically very stiff with low friction.

```
<ground_reactions>
  <contact type="BOGEY" name="NOSE">
    <location unit="IN">
      <x>-6.8</x> <y>0</y> <z>-19.5</z>
    </location>
    <static_friction> 0.80 </static_friction>
    <dynamic_friction> 0.50 </dynamic_friction>
    <rolling_friction> 0.02 </rolling_friction>
    <spring_coeff unit="LBS/FT"> 1800 </spring_coeff>
    <damping_coeff unit="LBS/FT/SEC"> 600 </damping_coeff>
    <damping_coeff_rebound unit="LBS/FT/SEC"> 1200 </damping_coeff_rebound>
    <max_steer unit="DEG"> 10 </max_steer>
    <brake_group> NONE </brake_group>
    <retractable>0</retractable>
  </contact>
  <contact type="BOGEY" name="LEFT_MAIN">
    <location unit="IN"> <x>58.2</x><y>-43</y><z>-15.5</z> </location>
    <static_friction>0.8</static_friction>
    <dynamic_friction>0.5</dynamic_friction>
    <rolling_friction>0.02</rolling_friction>
    <spring_coeff unit="LBS/FT"> 5400 </spring_coeff>
    <damping_coeff unit="LBS/FT/SEC"> 1600 </damping_coeff>
    <brake_group> LEFT </brake_group>
  </contact>
  <contact type="STRUCTURE" name="LEFT_WINGTIP">
    <location unit="IN"> <x>43.2</x><y>-214.8</y><z>59.4</z> </location>
    <static_friction>0.2</static_friction>
    <dynamic_friction>0.2</dynamic_friction>
    <spring_coeff unit="LBS/FT"> 20000 </spring_coeff>
    <damping_coeff unit="LBS/FT/SEC"> 2000 </damping_coeff>
  </contact>
</ground_reactions>
```

Friction forces are computed by a projected-Gauss-Seidel iterative Lagrange-multiplier solver (Catto 2005) inside `FGAccelerations::CalculateFrictionForces()`, so multi-wheel contacts respect the no-interpenetration and Coulomb-friction constraints simultaneously.

4.6 external_reactions: bolt-on forces

Anything that applies a force or moment that is neither aero, nor propulsion, nor ground contact goes here: parachutes, towlines, JATO bottles, magnetic launchers.

```
<external_reactions>
  <property>fcs/parachute_reef_pos_norm</property> <!-- declare -->

  <force name="parachute" frame="WIND">
    <function>
      <product>
        <property>aero/qbar-psf</property>
        <property>fcs/parachute_reef_pos_norm</property>
        <value>1.0</value>
        <value>10000.0</value> <!-- chute area in sq-ft -->
      </product>
    </function>
    <location unit="FT"> <x>1</x><y>0</y><z>0</z> </location>
    <direction> <x>-1</x><y>0</y><z>0</z> </direction>
  </force>
</external_reactions>
```

4.7 propulsion: engines, tanks, thrusters

Propulsion is two-tier: an *engine* XML file (loaded from \$JSBSIM_ROOT/engine/) describes the engine; the aircraft XML references it and adds a *thruster* (the body that applies the actual force) and one or more *tanks*. Engine types are:

- **piston_engine** — internal-combustion piston engine with power tables vs. RPM, throttle, mixture, MAP, altitude.
- **turbine_engine** — turbojet/turbofan with idle/military/AB tables.
- **turboprop_engine** — turboprop with shaft horsepower tables.
- **rocket_engine** — solid or liquid rocket with thrust/lsp.
- **electric_engine** — DC motor, power × throttle.
- **brushless_dc_motor** — torque vs. RPM curve.
- **rotor** — helicopter rotor (induced flow + flapping).

```
<propulsion>
  <engine file="eng_io320">
    <feed>0</feed>          <!-- consume from tank 0 -->
    <feed>1</feed>
    <thruster file="prop_75in2f">
      <location unit="IN"> <x>-37.7</x><y>0</y><z>26.6</z> </location>
      <orient unit="DEG">
        <pitch>0</pitch><roll>0</roll><yaw>0</yaw>
      </orient>
      <sense>1</sense>      <!-- 1 = right-hand prop -->
      <p_factor>5</p_factor> <!-- effective offset for P-factor -->
    </thruster>
  </engine>

  <tank type="FUEL" number="0">
    <location unit="IN"> <x>56</x><y>-112</y><z>59.4</z> </location>
    <type>AVGAS</type>
    <capacity unit="LBS"> 185 </capacity>
    <contents unit="LBS"> 100 </contents>
    <priority>1</priority>
  </tank>
  <tank type="FUEL" number="1">
    <location unit="IN"> <x>56</x><y> 112</y><z>59.4</z> </location>
    <capacity unit="LBS"> 185 </capacity>
    <contents unit="LBS"> 100 </contents>
  </tank>
</propulsion>
```

An example engine file (engine/eng_io320.xml):

```
<piston_engine name="IO320">
  <minmp unit="INHG"> 10.0 </minmp>
  <maxmp unit="INHG"> 28.5 </maxmp>
  <displacement unit="IN3"> 320.0 </displacement>
  <maxhp> 160.0 </maxhp>
  <bsfc> 0.32 </bsfc>
  <cycles> 4.0 </cycles>
  <idlerpm> 550.0 </idlerpm>
  <maxrpm> 2700.0 </maxrpm>
  <maxthrottle> 1.0 </maxthrottle>
  <minthrottle> 0.1 </minthrottle>
  <sparkfaildrop> 0.1 </sparkfaildrop>
</piston_engine>
```

4.8 flight_control: command shaping

The FCS is a directed graph of components arranged into named *channels*. Inputs come from pilot commands (fcs/elevator-cmd-norm, etc., set by the input device or script) and outputs flow into the aerodynamic deflection properties (fcs/elevator-pos-rad, etc.) that the aerodynamics functions read.

```

<flight_control name="FCS: c172">

  <channel name="Pitch">
    <summer name="Pitch Trim Sum">
      <input>fcs/elevator-cmd-norm</input>
      <input>fcs/pitch-trim-cmd-norm</input>
      <clipto> <min>-1</min><max>1</max> </clipto>
    </summer>

    <aerosurface_scale name="Elevator Control">
      <input>fcs/pitch-trim-sum</input>
      <gain>0.01745</gain>      <!-- deg -> rad -->
      <range> <min>-28</min><max>23</max> </range>
      <output>fcs/elevator-pos-rad</output>
    </aerosurface_scale>
  </channel>

  <channel name="Flaps">
    <kinematic name="Flaps Control">
      <input>fcs/flap-cmd-norm</input>
      <traverse>
        <setting><position>0</position> <time>0</time></setting>
        <setting><position>10</position><time>2</time></setting>
        <setting><position>20</position><time>2</time></setting>
        <setting><position>30</position><time>2</time></setting>
      </traverse>
      <output>fcs/flap-pos-deg</output>
    </kinematic>
  </channel>
</flight_control>

```

Available FCS components include `summer`, `pure_gain`, `scheduled_gain`, `aerosurface_scale`, `kinematic`, `deadband`, `switch`, `fcs_function`, `pid`, `lag_filter`, `lead_lag_filter`, `washout_filter`, `second_order_filter`, `integrator`, `sensor`, `actuator` and the generic `fcs_function` which lets you embed any function-language expression as a block. PIDs and filters discretise their continuous-time form using the simulation dt .

4.9 autopilot and system: same toolbox, different scope

`<autopilot>` and `<system>` use the same FCS components and channel structure. They are split out because they execute as separate sub-groups: typical practice is to put basic stick/pedal shaping in *flight_control*, classical autopilot loops (altitude-hold, heading-hold, autothrottle) in *autopilot*, and avionics or electrical-system logic in *system* (which may be loaded from external files in `$AC/Systems/`).

4.10 aerodynamics: the core of the model

Aerodynamics is large enough to warrant its own chapter — see Chapter 5. The skeleton is:

```

<aerodynamics>

  <alphalimits unit="RAD">          <!-- physical alpha clamp -->
    <min>-0.087</min>
    <max> 0.280</max>
  </alphalimits>

  <hysteresis_limits unit="RAD">    <!-- stall hysteresis -->
    <min>0.09</min>
    <max>0.36</max>
  </hysteresis_limits>

  <!-- Helper functions (often ground effect, induced velocity) -->
  <function name="aero/function/kCLge"> ... </function>

  <!-- Six axes: three forces and three moments -->
  <axis name="DRAG"> ... </axis>
  <axis name="SIDE"> ... </axis>
  <axis name="LIFT"> ... </axis>

```

```

    <axis name="ROLL"> ... </axis>
    <axis name="PITCH"> ... </axis>
    <axis name="YAW"> ... </axis>
</aerodynamics>

```

4.11 input and output

<input> enables an external interface, typically a TCP/UDP socket for telnet-style command/query interaction.

<output> is much more common: a logging configuration for CSV files, FlightGear binary, sockets, etc.

```

<output name="flight.csv" type="CSV" rate="60">
  <property> aero/qbar-psf </property>
  <property> velocities/vt-fps </property>
  <rates>      ON </rates>
  <velocities>ON </velocities>
  <forces>     ON </forces>
  <moments>    ON </moments>
  <position>   ON </position>
  <fcs>        OFF</fcs>
  <aerosurfaces>OFF</aerosurfaces>
</output>

<output type="FLIGHTGEAR" rate="60" protocol="UDP" port="5500"/>

<input port="1138"/>      <!-- telnet on TCP 1138 -->

```


Chapter 5: Aerodynamics — Deep Dive

5.1 The six axes and the function/table model

Inside <aerodynamics> the heart of the model is an arbitrary number of <function> elements grouped into <axis> blocks. JSBSim sums every function inside an axis to obtain that axis's total force or moment *in physical units* (lbf, ft-lbf). It does **not** do any additional non-dimensionalisation — you embed the $\bar{q} \cdot S$ factor yourself, by convention.

Axis names map to indices (FGAerodynamics.cpp:57-69):

Forces (axis index 0..2)	Moments (axis index 3..5)
0 DRAG	3 ROLL (l)
1 SIDE	4 PITCH (m)
2 LIFT	5 YAW (n)

Alternative force axes:	Alternative moment frames:
X, Y, Z (body)	frame="WIND"
AXIAL, NORMAL (axial/norm)	frame="STABILITY"

5.2 How a single function evaluates

Every function returns a scalar each tick. A canonical drag term looks like:

```
<function name="aero/coefficient/CD0">
  <description>Drag at zero lift</description>
  <product>
    <property>aero/qbar-psf</property>      <!-- q-bar -->
    <property>metrics/Sw-sqft</property>    <!-- S -->
    <value>0.027</value>                    <!-- CD0 -->
  </product>
</function>
```

$$F_{D,0} = C_{D0} \cdot \bar{q} \cdot S$$

Inside FGAerodynamics::Run() the values returned by all <axis name="DRAG"> children are summed into the DRAG slot of vFNative; the LIFT and SIDE slots are filled the same way; then a rotation by T_{w2b} lifts wind-axis forces into the body frame, the moments computed by ROLL/PITCH/YAW are added at the AERORP, and the moments are finally transferred to the CG by $M_{cg} = M_{arp} + r_{arp \rightarrow cg} \times F$.

5.3 The function language

JSBSim's function language is a small Lisp-ish XML expression tree. The leaves are <value> (constant) and <property> (reference to the property tree); the internal nodes are operators.

Arithmetic	sum, difference, product, quotient, pow, sqrt, abs, min, max, avg
Trigonometric	sin, cos, tan, asin, acos, atan, atan2
Exponential	exp, ln, log2, log10
Logical	lt, le, gt, ge, eq, nq, and, or, not, ifthen, switch
Modular	mod, floor, ceil, fmod, roundmultiple
Unit	toradians, todegrees
Random	random (Gaussian), urandom (uniform)
Constants	pi, value/v, integer
Table	table (1-D, 2-D, 3-D, n-D)
Property	property/p

A more interesting example — induced drag, $C_{Di} = C_L^2 / (\pi \cdot AR \cdot e)$ — turned into a JSBSim function:

```
<function name="aero/coefficient/CDi">
  <description>Induced drag</description>
  <product>
    <property>aero/qbar-psf</property>
    <property>metrics/Sw-sqft</property>
```

```

    <quotient>
      <pow>
        <property>aero/cl-squared</property>      <!-- CL^2 -->
        <value>0.5</value>                        <!-- *not* powered -->
      </pow>
      <product>
        <pi/>
        <property>metrics/aspectratio</property>
        <value>0.85</value>                      <!-- e -->
      </product>
    </quotient>
  </product>
</function>

```

5.4 Tables: 1-D, 2-D, 3-D, and higher

Tables are the workhorse of any non-trivial model — they let you encode the CFD-derived lookup of any coefficient against any combination of state variables. JSBSim uses **linear interpolation** in every dimension; outside the tabulated range the boundary value is clamped (no extrapolation), so it is your job to cover the operational envelope.

1-D table (a vector lookup against one variable):

```

<table>
  <independentVar lookup="row">aero/alpha-rad</independentVar>
  <tableData>
    -0.0900  -0.2200
     0.0000   0.2500
     0.0900   0.7300
     0.1500   1.0000
     0.2800   1.1500
  </tableData>
</table>

```

2-D table (e.g. drag vs alpha and flap angle):

```

<table>
  <independentVar lookup="row">aero/alpha-rad</independentVar>
  <independentVar lookup="column">fcs/flap-pos-deg</independentVar>
  <tableData>
           0.0    10.0    20.0    30.0
    -0.0873  0.0041  0.0000  0.0005  0.0014
    -0.0349  0.0003  0.0057  0.0108  0.0141
     0.0000  0.0052  0.0168  0.0251  0.0299
     0.0524  0.0240  0.0452  0.0583  0.0655
     0.1571  0.0962  0.1353  0.1573  0.1690
  </tableData>
</table>

```

3-D table (e.g. pitching moment vs alpha, elevator, Mach). You stack 2-D tables, each with a `breakPoint` attribute for the third dimension:

```

<table>
  <independentVar lookup="row">aero/alpha-rad</independentVar>
  <independentVar lookup="column">fcs/elevator-pos-rad</independentVar>
  <independentVar lookup="table">velocities/mach</independentVar>

  <tableData breakPoint="0.30">                  <!-- Mach = 0.30 -->
    -0.4  -0.2  0.0  0.2  0.4
    -0.1  0.20  0.11  0.04 -0.06 -0.16
     0.0  0.15  0.07  0.00 -0.07 -0.15
     0.1  0.11  0.04 -0.04 -0.12 -0.21
  </tableData>

  <tableData breakPoint="0.80">                  <!-- Mach = 0.80 -->
    -0.4  -0.2  0.0  0.2  0.4
    -0.1  0.18  0.10  0.03 -0.05 -0.14
     0.0  0.13  0.06  0.00 -0.06 -0.13
     0.1  0.10  0.03 -0.03 -0.10 -0.19
  </tableData>

```

</table>

Higher dimensions are supported recursively. In practice 3-D is the limit of what you should build by hand; for 4-D and above use a script to generate the XML from a CFD database.

5.5 Properties commonly used in aerodynamics

aero/qbar-psf	$q\text{-bar} = 0.5 \cdot \rho \cdot V_t^2$ (psf)
aero/alpha-rad	angle of attack (radians)
aero/beta-rad	sideslip angle (radians)
aero/alphadot-rad_sec	alpha-dot
aero/betadot-rad_sec	beta-dot
aero/mag-beta-rad	beta (useful in drag terms)
aero/bi2vel	$b / (2 V)$ --- roll-rate non-dim factor
aero/ci2vel	$cbar / (2 V)$ --- pitch-rate non-dim factor
aero/h_b-mac-ft	height above ground / chord, for ground effect
aero/cl-squared	CL^2 (computed from previous tick's lift)
velocities/p-aero-rad_sec	roll rate in aero frame (rad/s)
velocities/q-aero-rad_sec	pitch rate in aero frame
velocities/r-aero-rad_sec	yaw rate in aero frame
velocities/mach	Mach number
velocities/vt-fps	true airspeed (ft/s)
metrics/Sw-sqft	wing area
metrics/bw-ft	wing span
metrics/cbarw-ft	mean aerodynamic chord
fcs/elevator-pos-rad	elevator deflection from FCS output
fcs/aileron-pos-rad	aileron
fcs/rudder-pos-rad	rudder
fcs/flap-pos-deg	flap (commonly in degrees)
gear/gear-pos-norm	gear position (0..1)
fcs/speedbrake-pos-rad	speedbrake

The aero-frame angular rates p-aero, q-aero, r-aero are the body-frame rates rotated into the wind frame (rotated by α only — the stability frame). Always use these for the damping derivatives, not the raw p-rad_sec.

5.6 Non-dimensionalisation by hand

Because JSBSim does not non-dimensionalise for you, you wear the responsibility of writing the dimensional formula correctly. These are the textbook formulas you embed in <product>:

$$\text{Lift} = C_L \cdot \bar{q} \cdot S$$

$$\text{Drag} = C_D \cdot \bar{q} \cdot S$$

$$\text{Side} = C_Y \cdot \bar{q} \cdot S$$

$$\text{Roll moment} = C_l \cdot \bar{q} \cdot S \cdot b$$

$$\text{Pitch moment} = C_m \cdot \bar{q} \cdot S \cdot \bar{c}$$

$$\text{Yaw moment} = C_n \cdot \bar{q} \cdot S \cdot b$$

For damping derivatives multiply by the appropriate $b/(2V)$ or $\bar{c}/(2V)$ factor:

$$\text{Roll-rate moment} = C_{l_p} \cdot \bar{q} \cdot S \cdot b \cdot [b/(2V)] \cdot p$$

$$\text{Pitch-rate moment} = C_{m_q} \cdot \bar{q} \cdot S \cdot \bar{c} \cdot [\bar{c}/(2V)] \cdot q$$

$$\text{Yaw-rate moment} = C_{n_r} \cdot \bar{q} \cdot S \cdot b \cdot [b/(2V)] \cdot r$$

$$\dot{\alpha} \text{ moment} = C_{m_{\dot{\alpha}}} \cdot \bar{q} \cdot S \cdot \bar{c} \cdot [\bar{c}/(2V)] \cdot \dot{\alpha}$$

5.7 Worked example: rolling moment due to roll rate

From aircraft/c172p/c172p.xml:733:

```
<function name="aero/coefficient/Clp">
  <description>Roll moment due to roll rate (roll damping)</description>
  <product>
    <property>aero/qbar-psf</property>          <!-- q-bar -->
    <property>metrics/Sw-sqft</property>         <!-- S    -->
    <property>metrics/bw-ft</property>           <!-- b    -->
    <property>aero/bi2vel</property>             <!-- b/(2V) -->
    <property>velocities/p-aero-rad_sec</property> <!-- p    -->
    <value>-0.4840</value>                      <!-- Clp   -->
  </product>
</function>
```

C_{l_p} for the C-172 is constant at -0.484 — the wing's roll damping. Multiplying out gives a body-frame rolling moment in ft-lbf, which is what the ROLL axis expects.

5.8 Static derivatives — getting them from CFD

Static derivatives ($\partial C/\partial \alpha$, $\partial C/\partial \beta$, $\partial C/\partial \delta$) characterise the *steady* aerodynamic response of the aircraft to changes in α , β and control deflections. They are obtained from CFD by running the aircraft as a rigid body in steady-state at a grid of (α , β , δ , M) points and reading off the resulting forces and moments. The list below is a typical minimum sweep for a subsonic conventional aircraft:

Derivative	Independent vars	CFD sweep
$C_L(\alpha, M, \text{flap})$	α, M, flap	steady cruise over $\alpha \in [-6^\circ, 18^\circ]$, stalled = clip
$C_D(\alpha, M, \text{flap, gear})$	$\alpha, M, \text{flap, gear}$	Same matrix; $C_D = \text{drag}/\bar{q}S$
$C_m(\alpha, M, \delta_e)$	α, M, δ_e	α -sweep at each δ_e and Mach
$C_Y(\beta, \delta_r)$	β, δ_r	β -sweep $\in [-15^\circ, 15^\circ]$
$C_l(\beta, \alpha, \delta_a)$	β, α, δ_a	β -sweep + δ_a increments
$C_n(\beta, \alpha, \delta_r)$	β, α, δ_r	β -sweep + δ_r increments
$C_{L\alpha}$	α	slope of C_L - α curve
$C_{m\alpha}$	α	slope of C_m - α curve (negative $\rightarrow$ stable)
$C_{n\beta}$	β	slope of C_n - β curve (positive $\rightarrow$ stable)
$C_{l\beta}$	β	dihedral effect

5.9 Dynamic (damping) derivatives — running CFD with motion

Dynamic derivatives describe the moment created by a rotation rate (p , q , r , $\dot{\alpha}$). They cannot be obtained from a steady-state snapshot of the geometry; you need *moving* CFD or the classical engineering analogues.

- **Forced oscillation:** in URANS or LES, run the aircraft oscillating sinusoidally about the relevant axis at a known amplitude and reduced frequency. Extract the in-phase and out-of-phase moment components — these give the static and damping derivatives respectively. Industry-standard tools: ANSYS Fluent, OpenFOAM with overset, STAR-CCM+, NASA OVERFLOW.
- **Quasi-steady method:** run several steady CFD cases with a constant rotation rate p imposed on the inertial frame; extract the rolling moment from each. The slope $\Delta C_l / \Delta(p b / 2V)$ is C_{l_p} . Fast but limited to small angles.
- **Strip-theory / lifting-line:** a cheap engineering fallback. For early model bring-up, take handbook estimates (Roskam, Etkin, USAF DATCOM) — they are within $\pm 30\%$ of high-fidelity data for conventional aircraft.
- **System ID:** if you already have flight-test data, fit the damping derivatives by minimising the simulation residual on a doublet manoeuvre. This is what AIAA SciTech "flight-validated model" papers actually do.

Damping derivatives you should obtain at minimum:

Symbol	Meaning	Typical sign	Method
C_{lp}	Roll damping	negative	forced oscillation about X
C_{lr}	Roll due to yaw rate	positive	forced oscillation about Z
C_{mq}	Pitch damping	negative	forced oscillation about Y
$C_{m_{\dot{\alpha}}}$	Pitch due to $\dot{\alpha}$	negative	forced plunge oscillation
C_{nr}	Yaw damping	negative	forced oscillation about Z
C_{np}	Yaw due to roll	small, signed	forced oscillation about X
C_{Yp}	Side force due to roll	small	from same X-oscillation run
C_{Yr}	Side force due to yaw	small	from same Z-oscillation run

5.10 Control derivatives

Control derivatives ($C_{m_{\delta e}}$, $C_{l_{\delta a}}$, $C_{n_{\delta r}}$, ...) are the slopes of moment vs. control deflection. They are static, so you obtain them in the same steady CFD batch as the static derivatives, simply by including control deflection as a sweep axis. For non-linear control authority (very common at high α or high deflection) prefer a 2-D table (α , δ) over a single coefficient.

```
<function name="aero/coefficient/Cmde">
  <description>Pitch moment due to elevator</description>
  <product>
    <property>aero/qbar-psf</property>
    <property>metrics/Sw-sqft</property>
    <property>metrics/cbarw-ft</property>
    <table>
      <independentVar lookup="row">aero/alpha-rad</independentVar>
      <independentVar lookup="column">fcs/elevator-pos-rad</independentVar>
      <tableData>
        -0.40  -0.20   0.00   0.20   0.40
        -0.10   0.22   0.11   0.00  -0.11  -0.22
         0.00   0.21   0.10   0.00  -0.10  -0.21
         0.10   0.18   0.09   0.00  -0.09  -0.18
         0.20   0.12   0.06   0.00  -0.06  -0.12
      </tableData>
    </table>
  </product>
</function>
```

5.11 Ground effect and Reynolds effects

Ground effect is conventionally implemented as a *multiplier* on lift and drag, evaluated as a 1-D table against height-above-ground normalised by mean chord (aero/h_b-mac-ft):

```
<function name="aero/function/kCLge">
  <description>Lift multiplier due to ground effect</description>
  <table>
    <independentVar>aero/h_b-mac-ft</independentVar>
    <tableData>
      0.00  1.203
      0.10  1.127
      0.20  1.073
      0.50  1.019
      1.00  1.002
      1.10  1.000
    </tableData>
  </table>
</function>
```

Use aero/function/kCLge as a factor in the lift function. The same approach handles Mach effects (table vs. velocities/mach) and Reynolds (table vs. velocities/reynolds) if you have the CFD data to populate them.

5.12 Stall and stall hysteresis

<alphaLimits> clips the *physical* α used in aero calculations. <hysteresisLimits> defines a switching band — JSBSim exposes `aero/stall-hyst-norm` which is 0 below the low limit and 1 above the high limit. Use it as a second axis of a lift table to encode the difference between the un-stalled and stalled lift curves and get correct stall-recovery behaviour.

Chapter 6: The Equations of Motion Under the Hood

All the XML and tables eventually feed into a small set of ordinary differential equations. This chapter walks the math as JSBSim implements it.

6.1 State vector

```
VehicleState { // FGPropagate.h:100
  FGLocation      vLocation; // ECEF position (ft)
  FGColumnVector3 vUVW; // body-frame velocity rel. ECEF
  FGColumnVector3 vPQR; // body-frame angular rate rel. ECEF
  FGColumnVector3 vPQRi; // body-frame angular rate rel. ECI
  FGQuaternion qAttitudeLocal; // body w.r.t. local NED
  FGQuaternion qAttitudeECI; // body w.r.t. ECI <-- primary
  FGColumnVector3 vInertialVelocity; // ECI frame velocity
  FGColumnVector3 vInertialPosition; // ECI frame position
  // ... history dequeues for multi-step integrators
};
```

6.2 Quaternion attitude kinematics

Attitude is propagated as the quaternion $q_{i \rightarrow b}$ from inertial to body. The time derivative is the kinematic equation

$$\dot{q} = \frac{1}{2} \cdot q \otimes \omega_{b/i}$$

where $\omega_{b/i}$ is the angular velocity of the body with respect to the inertial frame, expressed in the body frame (vPQRi). JSBSim's `FGQuaternion::GetQDot()` computes exactly this; `FGPropagate::CalculateQuatdot()` wraps it. Multiple integrators are available:

```
0 eNone          freeze
1 eRectEuler      q(n+1) = q(n) + dt * q-dot(n)
2 eTrapezoidal    q(n+1) = q(n) + 0.5*dt*(q-dot(n)+q-dot(n-1))
3 eAdamsBashforth2 q(n+1) = q(n) + dt*(1.5 q-dot(n) - 0.5 q-dot(n-1))
4 eAdamsBashforth3 23/12, -16/12, +5/12 weights
5 eAdamsBashforth4 55/24, -59/24, 37/24, -9/24
6 eAdamsBashforth5 classical 5-step coefficients
7 eBuss1          q(n+1) = q(n) * exp(0.5*dt*omega)
8 eBuss2          augmented with omega-dot terms
9 eLocalLinearization Barker et al. (NASA TN D-7347)
```

Default integrators are rectangular Euler for attitude and angular rate, Adams-Bashforth-2 for translational velocity, Adams-Bashforth-3 for position. The Buss integrators are interesting because they preserve quaternion norm exactly (no need for the renormalisation that rectangular Euler requires every tick).

6.3 Translational dynamics in the inertial frame

JSBSim solves Newton's second law in the inertial frame and transforms back to body coordinates for output:

$$\dot{v}_{\text{body}} = F/m - (p_{\text{body}} + 2 \cdot T_{i2b} \cdot \Omega_{\text{planet}}) \times v_{\text{body}} - T_{i2b} \cdot \Omega_{\text{planet}} \times (\Omega_{\text{planet}} \times r_i)$$

The first term is the conventional $F = ma$. The second is the Coriolis acceleration, including the contribution from Earth's rotation ($\Omega_{\text{planet}} \approx 7.292 \cdot 10^{-5}$ rad/s about the ECI Z axis). The third is the centrifugal acceleration from rotating about the Earth's centre. For terrestrial flight at transport-aircraft scales these are small but visible — they are what makes the simulator agree with the NASA-2015 check-cases.

6.4 Rotational dynamics

$$J \cdot \dot{\omega}_i = M - \omega_i \times (J \cdot \omega_i)$$

The term $\omega \times (J\omega)$ is the Euler gyroscopic torque (non-zero whenever the inertia tensor is not isotropic). `FGAccelerations::CalculatePQRdot()` computes this directly. The inertia tensor J in the body frame is maintained by `FGMassBalance` and is updated every tick to account for fuel burn and pointmass shifts. An optional gravity-gradient torque is enabled via the property `simulation/gravitational-torque`; useful for orbital mechanics.

6.5 Total force and moment assembly

`FGAircraft` sums forces and moments from *aero*, *propulsion*, *ground*, *external*, *buoyant* models and exposes them as a single body-frame F and M to the `Accelerations` model. Gravity is treated separately by `FGInertial` (either spherical or WGS-84 with the J_2 term) and added directly to the acceleration vector — keeping it out of M means the moment of gravity about the CG is automatically zero (gravity acts at the CG by definition).

6.6 Ground friction as a constrained problem

Standard explicit integration of stiff spring-damper landing-gear models is unstable. JSBSim instead poses the friction problem as a *linear-complementarity problem (LCP)*: each contact point imposes a non-penetration constraint normal to the runway and a Coulomb-friction constraint tangent to it. The unknowns are the Lagrange multipliers λ . The system $A \cdot \lambda = b$ with $A = J \cdot M^{-1} \cdot J^T$ is solved iteratively (Projected Gauss-Seidel, Catto 2005) up to 50 iterations per tick — see `FGAccelerations::CalculateFrictionForces()`. The contact friction forces and the moment $r \times F$ are then added to the F and M vectors.

6.7 Trim

Trim is implemented in `FGTrim` by a bracketed root-finder applied independently to each axis-control pair, iterated in an outer loop until all axes converge. It is **not** Newton-Raphson — JSBSim uses a linear-interpolation secant variant with a 0.9 relaxation factor to suppress oscillation. The modes are:

Mode	Constraints	Controls
tLongitudinal	$\dot{w}=0, \dot{u}=0, \dot{q}=0$	α , throttle, elevator
tFull	longitudinal + $\dot{v}=0, \dot{r}=0, \dot{p}=0 + \psi$ track	ϕ , aileron, rudder, β
tGround	$\dot{w}=0, \dot{q}=0, \dot{r}=0$	altitude, θ , ϕ
tPullup	longitudinal at target load factor	α , throttle, elev
tTurn	coordinated turn at target bank angle	throttle, elevator, rudder
tCustom	user-defined (state, control) pairs	any

6.8 Atmospheric model

`FGStandardAtmosphere` implements the 1976 U.S. Standard Atmosphere by piecewise linear lapse rates from 0 to 86 km. Properties are exposed under `atmosphere/`: T -R, P -psf, ρ -slugs/ft³, a -fps. A custom atmosphere can be subclassed; the C-172 model offsets temperature to model density altitude effects.

6.9 Winds and turbulence

`FGWinds` supports:

- Steady wind in NED (set `atmosphere/wind-north-fps`, `wind-east-fps`, `wind-down-fps`).
- Linear wind shear with altitude.
- Gusts (1-cosine "discrete gust" model used in MIL-F-8785C).
- Dryden or von Kármán continuous turbulence with configurable intensity and scale lengths.
- Microburst model with three components and a vortex ring.

`FGAuxiliary` consumes the wind vector to compute the airmass-relative velocity used by aerodynamics.

6.10 Property tree

FGPropertyManager wraps SimGear's property tree. Every state variable, control input, FCS signal, aero coefficient and atmosphere quantity is exposed at a string path; values flow between models entirely through it. Aircraft XML <property> elements can *declare* a new property (creating it if needed); FCS components, aero functions and external interfaces all bind themselves to the tree.

```
# Inspect a property in the JSBSim Python module
import jsbsim
fdm = jsbsim.FGFDMExec(None)
fdm.load_model("c172p")
fdm.run_ic()
print(fdm["velocities/vt-fps"], fdm["aero/alpha-deg"])
```

Chapter 7: Building Your Own Aircraft, Step-by-Step

This is the practical playbook. We assume you have access to **geometry** (CAD or at least 3-view), **mass properties** (weight, CG, ideally an inertia estimate), **engine data** (power or thrust curves), and **aerodynamic data** (CFD or wind tunnel). The workflow below brings them together into a working JSBSim model in a couple of weeks of part-time effort.

7.1 Step 1: Lay out the directory

```
$JSBSIM_ROOT/aircraft/MyAcft/
  MyAcft.xml           # the main XML (this is what JSBSim loads)
  reset00.xml          # initial conditions for default launch
  Systems/             # (optional) external <system> files
  Engines/             # (optional) aircraft-specific engine files
$JSBSIM_ROOT/scripts/
  MyAcft_takeoff.xml   # a script that loads MyAcft + reset00
```

Engines and propellers can live in the project-wide `$JSBSIM_ROOT/engine/` directory if you want to reuse them across multiple airframes.

7.2 Step 2: Skeleton XML

```
<?xml version="1.0"?>
<fdm_config name="MyAcft" version="2.0" release="ALPHA"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="http://jsbsim.sourceforge.net/JSBSim.xsd">

  <fileheader>
    <author>You</author>
    <filecreationdate>2026-05-24</filecreationdate>
    <description>My new aircraft.</description>
  </fileheader>

  <metrics>           ... </metrics>
  <mass_balance>      ... </mass_balance>
  <ground_reactions>  ... </ground_reactions>
  <propulsion>        ... </propulsion>
  <flight_control name="FCS: MyAcft">
    <channel name="Pitch"> ... </channel>
    <channel name="Roll">  ... </channel>
    <channel name="Yaw">   ... </channel>
  </flight_control>
  <aerodynamics>      ... </aerodynamics>
  <output type="CSV" rate="60" name="MyAcft.csv">
    <rates>ON</rates>
    <velocities>ON</velocities>
    <position>ON</position>
  </output>
</fdm_config>
```

7.3 Step 3: Geometry, mass, and gear

Geometry: from your CAD, pull out the wing reference area S , wing span b , mean aerodynamic chord $\bar{c}$, tail areas and arms. Pick the AERORP at 25% MAC and locate it in your structural frame. Locate the CG (any acceptable balance envelope point — you can shift it later with pointmasses).

Mass: empty weight + crew/passengers/cargo as pointmasses. For inertia, if you have a CAD model the easiest path is to export it to a kinematics tool (SolidWorks, Onshape, Fusion 360) that gives you $I_{xx,yy,zz,xy,xz,yz}$ about the CG directly. Otherwise estimate from radii of gyration (Roskam vol. V tables 9.1-9.4).

Gear: locate each contact point in the structural frame. Spring/damper choice is part craft, part calculation: aim for static deflection of 2-4 inches under weight on wheels, and a damping ratio of about 0.3-0.5. A quick estimator:

$$k = W_{\text{gear}} / \Delta_{\text{static}} \quad c = 2 \zeta \sqrt{(k \cdot W_{\text{gear}} / g)}$$

7.4 Step 4: Propulsion

Pick an engine file that matches your type. The standard JSBSim engine library has good starting points: eng_io320 (160 hp 4-cyl), CFM56 (turbofan), F100-PW-229 (afterburning turbofan), Estes_E9 (model rocket), DJI_E305 (drone electric). For a new engine, copy an existing file and edit displacement / max power / BSFC / RPM range. Pair the engine with a thruster — direct for jets/rockets, prop_* for propellers.

```
<propulsion>
  <engine file="eng_io360">                                <!-- 180 hp -->
    <feed>0</feed>
    <feed>1</feed>
    <thruster file="prop_77in">
      <location unit="IN"> <x>-30</x><y>0</y><z>2</z> </location>
      <orient unit="DEG"><pitch>2</pitch></orient> <!-- thrust line up 2° -->
    </thruster>
  </engine>
  <tank type="FUEL" number="0">
    <location unit="IN"><x>0</x><y>-80</y><z>30</z></location>
    <capacity unit="LBS">300</capacity>
    <contents unit="LBS">280</contents>
  </tank>
  <tank type="FUEL" number="1">
    <location unit="IN"><x>0</x><y>80</y><z>30</z></location>
    <capacity unit="LBS">300</capacity>
    <contents unit="LBS">280</contents>
  </tank>
</propulsion>
```

7.5 Step 5: Flight control system

The smallest possible FCS just maps fcs/elevator-cmd-norm (-1..+1) to fcs/elevator-pos-rad via an aerosurface_scale. Real aircraft add trim sums, rate limits, deadbands, gust filters, autopilots — but start minimal, get the aircraft flying open-loop, then add complexity.

7.6 Step 6: Aerodynamics from CFD

This is the most labour-intensive step. The general procedure:

- **1. Define your sweep grid.** A reasonable subsonic grid is $\alpha \in \{-6, -4, -2, 0, 2, 4, 6, 8, 10, 12, 14, 16, 18\}$ deg, $\beta \in \{-15, -10, -5, 0, 5, 10, 15\}$ deg, $\text{Mach} \in \{0.15, 0.30, 0.50, 0.70\}$, control deflections at 5-7 points each. That is ~3000 CFD cases for a full lookup matrix; in practice you decouple.
- **2. Run steady RANS cases** in your CFD tool of choice (OpenFOAM, Fluent, STAR-CCM+, SU2). For each case extract $F_x, F_y, F_z, M_x, M_y, M_z$ in the body frame about the AERORP and convert to coefficients by dividing by $\bar{q} \cdot S, \bar{q} \cdot S \cdot b, \bar{q} \cdot S \cdot \bar{c}$.
- **3. Decouple where possible.** Generally take C_L, C_D, C_m from the α -sweep at $\beta=0$; take C_Y, C_l, C_n from β -sweeps at $\alpha=0$; take control derivatives as the slope of moment vs. deflection at the trim condition. For high- α or transonic models the coupling is real and you do need a full 2-D or 3-D table.
- **4. Run damping derivative cases** — either forced oscillation or quasi-steady rolling/yawing/pitching with constant rate. Extract $C_{lp}, C_{mq}, C_{nr}, C_{lr}, C_{np}, C_{m_{\dot{\alpha}}}$.
- **5. Tabulate and validate.** Plot every coefficient as a function of every independent variable. Look for non-monotonic behaviour you didn't expect — usually a sign of mesh or convergence problems.
- **6. Convert to JSBSim XML.** The format is a simple matrix of numbers in <tableData>. The Python script in the next chapter automates this.

7.7 Step 7: Initial conditions and a smoke-test script

```

<!-- reset00.xml -->
<?xml version="1.0"?>
<initialize name="reset00">
  <ubody unit="FT/SEC"> 150 </ubody>      <!-- ~100 KT -->
  <vbody unit="FT/SEC"> 0 </vbody>
  <wbody unit="FT/SEC"> 0 </wbody>
  <latitude unit="DEG"> 37.6 </latitude>
  <longitude unit="DEG">-122.4</longitude>
  <altitude unit="FT"> 5000 </altitude>
  <phi unit="DEG"> 0 </phi>
  <theta unit="DEG">2 </theta>
  <psi unit="DEG"> 90 </psi>              <!-- heading east -->
</initialize>

<!-- scripts/MyAcft_smoke.xml -->
<?xml version="1.0"?>
<runscript name="MyAcft smoke">
  <use aircraft="MyAcft" initialize="reset00"/>
  <run start="0" end="120" dt="0.00833333">
    <event name="Trim">
      <condition>simulation/sim-time-sec ge 0</condition>
      <set name="simulation/do_simple_trim" value="1"/>
      <notify/>
    </event>
    <event name="Pitch doublet">
      <condition>simulation/sim-time-sec ge 30</condition>
      <set name="fcs/elevator-cmd-norm" value="0.2" type="FG_VALUE"/>
      <notify/>
    </event>
    <event name="Pitch return">
      <condition>simulation/sim-time-sec ge 32</condition>
      <set name="fcs/elevator-cmd-norm" value="0"/>
      <notify/>
    </event>
  </run>
</runscript>

```

Run it with:

```
JSBSim --script=scripts/MyAcft_smoke.xml --logdirectivefile=...
```

7.8 Step 8: Iteration loop

- **Does it trim?** If `do_simple_trim` fails, check sign conventions (elevator producing the right direction of pitching moment), check that $C_{m_\alpha} < 0$ (statically stable in pitch), and check the α range of your tables covers the trim α .
- **Does it fly straight?** Static lateral-directional stability needs $C_{n_\beta} > 0$ and $C_{l_\beta} < 0$. If the aircraft rolls off, check the dihedral effect.
- **Does it respond like the real aircraft?** Compare phugoid, short-period, Dutch-roll, roll-mode and spiral-mode eigenvalues against published data or flight test.
- **Does it land?** Tune gear spring/damping until the bounce looks right. If the aircraft sinks through the runway, your spring is too soft or the contact point is too high (Z too positive in structural frame).

Chapter 8: CFD-to-JSBSim Workflow

This chapter describes the recommended workflow for generating JSBSim aerodynamic tables from CFD data, including the specific cases to run, the formulas to apply, and a reference Python script to convert results into JSBSim XML.

8.1 Choosing a CFD solver

- **OpenFOAM** (open source) — *simpleFoam* for steady RANS; *pimpleFoam* with overset for moving meshes. Free but mesh quality and turbulence model selection are on you.
- **SU2** (open source) — modern, adjoint capable, good for shape optimisation as well as analysis.
- **ANSYS Fluent / STAR-CCM+** — commercial, very mature. Worth the licence if you have it.
- **NASA OVERFLOW / FUN3D** — research-grade structured/unstructured RANS, available to U.S. universities.
- **XFLR5 / AVL** (vortex-lattice) — fast, surprisingly accurate for early-design subsonic estimates. Use it to bootstrap before spending CFD budget.

8.2 Recommended case matrix

For a conventional subsonic aircraft, a complete model needs the cases below. Each is a separate CFD run.

Group	Sweep	α (deg)	β (deg)	Output
Longitudinal static	α at clean config, $\beta=0$	−6 to +18 step 2	0	$C_L(\alpha)$, $C_D(\alpha)$, $C_m(\alpha)$
Longitudinal w/ flap	$\alpha \times \text{flap}$	−4 to +14 step 2	0	ΔC_L , ΔC_D , ΔC_m vs flap
Longitudinal w/ elevator	$\alpha \times \delta_e$	−4 to +14 step 2	0	$C_{m_{\delta e}}(\alpha, \delta_e)$
Lateral static	β at α_{cruise}	α_{cr}	−15 to +15 step 3	$C_Y(\beta)$, $C_l(\beta)$, $C_n(\beta)$
Aileron	$\beta \times \delta_a$	α_{cr}	−10 to +10	$C_{l_{\delta a}}$, $C_{n_{\delta a}}$
Rudder	$\beta \times \delta_r$	α_{cr}	−10 to +10	$C_{Y_{\delta r}}$, $C_{n_{\delta r}}$
Compressibility	Mach	0	0	$\Delta C_D(M)$, $\Delta C_m(M)$
Ground effect	$h/\bar{c}$	α_{cr}	0	$k_{CL,ge}(h/\bar{c})$, $k_{CD,ge}(h/\bar{c})$
Pitch damping	forced osc. about Y	α_{cr}	0	C_{mq} , $C_{m_{\dot{\alpha}}}$
Roll damping	forced osc. about X	α_{cr}	0	C_{lp} , C_{Yp} , C_{np}
Yaw damping	forced osc. about Z	α_{cr}	0	C_{nr} , C_{Yr} , C_{lr}

If you have transonic or supersonic operations, add Mach as a third axis to all of the longitudinal cases. For a fighter aircraft or a UCAV operating to high α , double the α range and step size.

8.3 Forced-oscillation method for damping derivatives

The cleanest way to get C_{mq} et al. from CFD is to drive the geometry sinusoidally about the relevant axis with small amplitude and a known reduced frequency, then fit the in-phase and out-of-phase moment components. For pitch damping:

$$\theta(t) = \theta_0 + \Delta\theta \cdot \sin(\omega t)$$

$$q(t) = \Delta\theta \cdot \omega \cdot \cos(\omega t)$$

$$C_m(t) = C_{m,0} + C_{m_{\alpha}} \cdot \Delta\alpha \cdot \sin(\omega t) + [C_{mq} + C_{m_{\dot{\alpha}}}] \cdot (\bar{c}/2V) \cdot \Delta\theta \cdot \omega \cdot \cos(\omega t)$$

Fit by least squares: the cos-coefficient is $(C_{mq} + C_{m_{\dot{\alpha}}}) \cdot (\bar{c}/(2V)) \cdot \Delta\theta \cdot \omega$, the sin-coefficient is $C_{m_{\alpha}} \cdot \Delta\alpha$. Separating C_{mq} from $C_{m_{\dot{\alpha}}}$ requires an additional *plunge* oscillation (α changes without q changing). In practice many handbooks just publish the sum $(C_{mq} + C_{m_{\dot{\alpha}}})$ and split it roughly 70/30.

Recommended amplitudes and reduced frequencies for a transport: $\Delta\theta = 1^\circ$, $k = \omega\bar{c}/(2V) \in [0.01, 0.1]$. Five or ten cycles in CFD before extracting the fit.

8.4 Quasi-steady method (faster)

Run several steady CFD cases with a *constant* body-fixed angular rate applied as a rotating reference frame. Plot the resulting moment against $pb/(2V)$ (or $q\bar{c}/(2V)$, or $rb/(2V)$); the slope is the damping derivative. This is much cheaper than URANS but only valid for small rates and incompressible flow. Adequate for a first-cut model.

8.5 Extracting and tabulating

Each CFD case gives you body-frame F and M about a known point. Convert to body-frame coefficients about the AERORP, then rotate force coefficients into wind-axis if your JSBSim model uses LIFT/DRAG/SIDE:

$$C_L = C_Z \cos \alpha - C_X \sin \alpha$$

$$C_D = -C_X \cos \alpha - C_Z \sin \alpha$$

$$C_Y \text{ (unchanged from body)}$$

8.6 Python helper: CFD CSV → JSBSim XML

A minimal generator that turns a CSV with columns `alpha_deg`, `flap_deg`, `CL`, `CD`, `Cm` into a JSBSim LIFT/DRAG/PITCH function block:

```
import pandas as pd

df = pd.read_csv('cfd_results.csv')
alphas = sorted(df.alpha_deg.unique())
flaps = sorted(df.flap_deg.unique())

def emit_table(name, indvar_row, indvar_col, rows, cols, values):
    out = ['<table>',
          f' <independentVar lookup="row">{indvar_row}</independentVar>',
          f' <independentVar lookup="column">{indvar_col}</independentVar>',
          ' <tableData>']
    header = ' ' + ' '.join(f'{c:8.3f}' for c in cols)
    out.append(header)
    for r in rows:
        row = [f'{r:8.4f}'] + [f'{values[(r,c)]:8.4f}' for c in cols]
        out.append(' ' + ' '.join(row))
    out += [' </tableData>', '</table>']
    return '\n'.join(out)

CL = {(r,c): df[(df.alpha_deg==r)&(df.flap_deg==c)].CL.iloc[0]
      for r in alphas for c in flaps}

print('<function name="aero/coefficient/CL">')
print(' <product>')
print(' <property>aero/qbar-psf</property>')
print(' <property>metrics/Sw-sqft</property>')
alpha_rad = sorted([a*3.14159/180 for a in alphas])
print(emit_table('CL', 'aero/alpha-rad', 'fcs/flap-pos-deg',
                alpha_rad, flaps, CL))
print(' </product>')
print('</function>')
```

8.7 Validation: from CFD numbers to flight feel

Three classes of validation, in order of increasing difficulty:

- **Coefficient plots** — overlay JSBSim coefficients against the CFD source data. Should match by construction; if not, you have a units bug.
- **Eigenvalue analysis** — linearise the JSBSim model at cruise with the bundled `python/JSBSim/utils/linearize` tool. Compare phugoid, short-period, Dutch-roll, roll, spiral roots against textbook estimates (Stevens-Lewis ch. 5) or published data for similar aircraft.
- **Manoeuvre matching** — run pitch doublets, aileron rolls, rudder kicks. Plot p , q , r , α , β . Compare against flight test or documented handling qualities (Cooper-Harper, MIL-F-8785C).

Chapter 9: Reference Appendices

9.1 Property tree cheat sheet

```
# Pilot inputs (writable from any external program / script)
fcs/elevator-cmd-norm      -1..+1, positive = nose-up command
fcs/aileron-cmd-norm       -1..+1, positive = roll right
fcs/rudder-cmd-norm        -1..+1, positive = yaw right
fcs/throttle-cmd-norm[n]   0..1 per engine
fcs/mixture-cmd-norm[n]    0..1 per piston engine
gear/gear-cmd-norm         0 or 1
fcs/flap-cmd-norm          0..1
fcs/speedbrake-cmd-norm    0..1

# Aero state
aero/alpha-rad / alpha-deg
aero/beta-rad / beta-deg
aero/qbar-psf
velocities/vt-fps          true airspeed
velocities/vc-kts          calibrated airspeed
velocities/mach
velocities/p-rad_sec       body-frame angular rates
velocities/p-aero-rad_sec  aero-frame angular rates

# Position and attitude
position/lat-geod-deg, position/long-gc-deg
position/h-sl-ft           altitude above MSL
position/h-agl-ft          altitude above ground
attitude/phi-rad, theta-rad, psi-rad

# Forces and moments (body frame, lbf, ft-lbf)
forces/fbx-aero, fby-aero, fbz-aero
forces/fbx-prop, fby-prop, fbz-prop
moments/l-aero, m-aero, n-aero
moments/l-total, m-total, n-total
```

9.2 Common FCS components

Component	Purpose
summer	Sum inputs with optional bias and clipping.
pure_gain	$y = k * x$ (k can be a property)
scheduled_gain	$k = \text{table}(\text{independent_var})$; $y = k * x$
aerosurface_scale	Map normalised input to physical deflection range.
kinematic	Discrete positions with traverse times — flap, gear.
lag_filter	First-order lag: $\dot{y} = (x - y) / \tau$
lead_lag_filter	Continuous $(a\dot{y} + a\ddot{y}s) / (b\dot{y} + b\ddot{y}s)$
washout_filter	High-pass: $\tau s / (\tau s + 1)$
second_order_filter	Notch/low-pass with 4 numerator + 4 denominator coefficients
integrator	$y = \int x \, dt$, with optional reset trigger
deadband	Output zero within a band around input
switch	Conditional logic with test/default cases
fcs_function	Embed an arbitrary function expression as an FCS block
pid	PID(K_p, K_i, K_d) with trigger, optional clipto
sensor	Adds noise, bias, drift, lag, delay
actuator	Rate limit, lag, hysteresis, deadband, fail modes

9.3 Engine type cross-reference

Engine class	XML root tag	Typical thruster	Key inputs
Piston			throttle, mixture, magnetos
Turbojet/Turbofan		or	throttle, AB on/off
Turboprop			throttle, propeller pitch
Rocket			throttle (often 0/1)
Electric DC			throttle (PWM)
Brushless DC			throttle, current limit
Rotor		(integrated)	collective, cyclic, throttle

9.4 Source-code map

```

src/FGFDMEExec.cpp / .h          main executive, model orchestration
src/initialization/FGInitialCondition.cpp      IC parsing and application
src/initialization/FGTrim.cpp                  trim algorithm
src/input_output/FGPropertyManager.cpp         property tree bindings
src/input_output/FGInputType*.cpp              telnet/socket/QtJSBSim input drivers
src/input_output/FGOutputType*.cpp             CSV, FlightGear, socket output drivers
src/math/FGFunction.cpp                       function-language evaluator
src/math/FGTable.cpp                          N-D table interpolation
src/math/FGPropertyValue.cpp                  property-bound value node
src/math/FGQuaternion.cpp                     quaternion arithmetic
src/math/FGMatrix33.cpp                       3x3 matrix
src/math/FGColumnVector3.cpp                  3-vector
src/models/FGPropagate.cpp                     state integration
src/models/FGAccelerations.cpp                 Newton-Euler, ground LCP
src/models/FGAerodynamics.cpp                 aero force/moment assembly
src/models/FGAuxiliary.cpp                     alpha, beta, qbar, Mach
src/models/FGAtmosphere.cpp                     standard atmosphere
src/models/atmosphere/FGWinds.cpp              winds, gusts, turbulence
src/models/FGInertial.cpp                       gravity, WGS-84, planet rotation
src/models/FGMassBalance.cpp                   mass/CG/inertia composition
src/models/FGPropulsion.cpp                     engine container
src/models/propulsion/FGPiston.cpp              piston engine
src/models/propulsion/FGTurbine.cpp             turbojet/turbofan
src/models/propulsion/FGTurboProp.cpp          turboprop
src/models/propulsion/FGRocket.cpp              rocket
src/models/propulsion/FGElectric.cpp            electric
src/models/propulsion/FGBrushLessDCMotor.cpp   propeller
src/models/propulsion/FGPropeller.cpp           rocket nozzle
src/models/propulsion/FGNozzle.cpp              helicopter rotor
src/models/propulsion/FGRotor.cpp               fuel tank
src/models/propulsion/FGTank.cpp
src/models/FGGroundReactions.cpp               ground reactions container
src/models/FGLGear.cpp                         landing gear / contact point
src/models/flight_control/...                  FCS components (summer, pid, ...)
src/models/FGFCS.cpp                           flight control system orchestrator

```

9.5 Bibliography and references

- Berndt, J. S. *JSBSim Reference Manual* (online and PDF).
<https://jsbsim.sourceforge.net/documentation.html>
- Stevens B. L., Lewis F. L. *Aircraft Control and Simulation*, 2nd ed., Wiley, 2003. — definitive reference for the equations of motion JSBSim implements.
- Etkin B., Reid L. D. *Dynamics of Flight: Stability and Control*, 3rd ed., Wiley, 1996. — classical stability derivative definitions.

- Roskam, J. *Airplane Design, Vols. I-VIII*, DARcorp. — engineering estimates for inertia, gear, control-surface geometry.
- USAF DATCOM, *USAF Stability and Control DATCOM*, USAF AFFDL-TR-79-3032. — handbook estimates for stability and damping derivatives; useful sanity check on CFD.
- Cooke J., Zyda M., Pratt D., McGhee R. *NPSNET: Flight Simulation Dynamic Modeling Using Quaternions*, 1994.
- Buss S. *Accurate and Efficient Simulation of Rigid Body Rotations*, UCSD, 1999.
- Catto E. *Iterative Dynamics with Temporal Coherence*, Crystal Dynamics, 2005.
- U.S. Standard Atmosphere, 1976, NASA-TM-X-74335.
- NASA-NESC. *2015 Flight Simulation Check Cases*. <https://nescacademy.nasa.gov/flightsim/2015>
- MIL-F-8785C, *Military Specification, Flying Qualities of Piloted Airplanes*.
- MIL-HDBK-516B, *Department of Defense Handbook: Airworthiness Certification Criteria*.

9.6 Closing thoughts

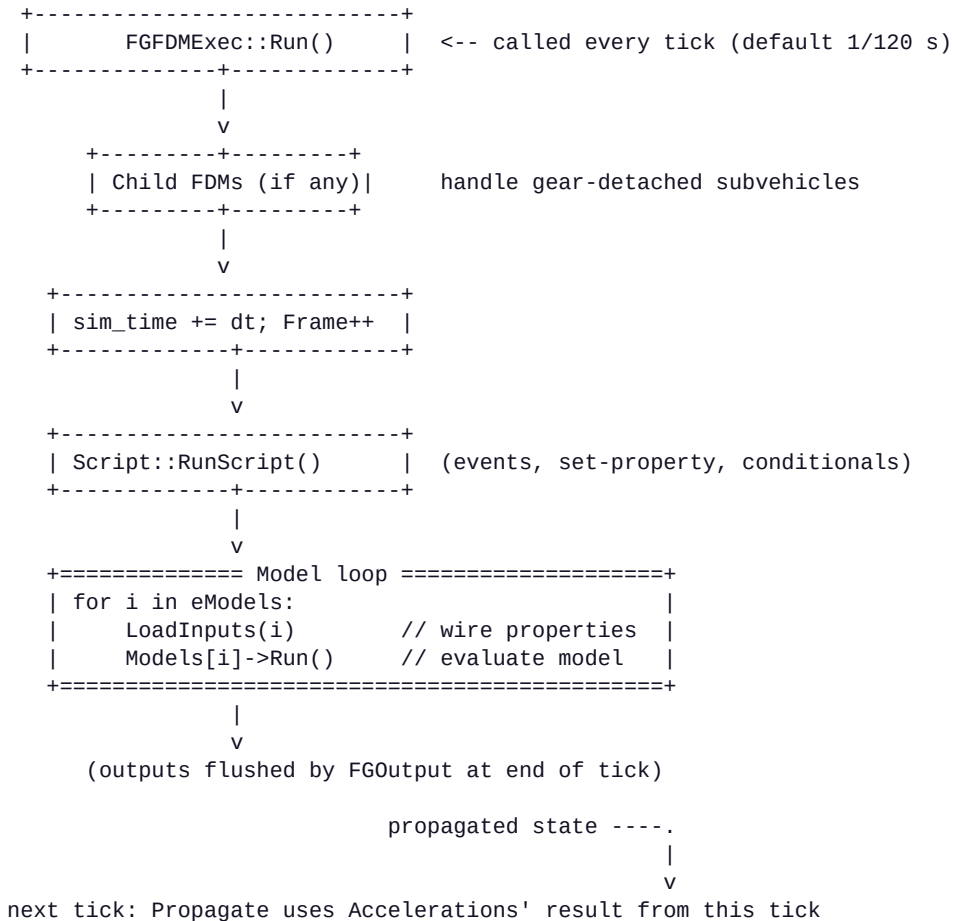
JSBSim's design rewards engineers who already think in terms of stability derivatives, frames and equations of motion: there is very little hidden magic. Most of what feels like rote configuration in the XML is actually a direct expression of the physics — a function multiplying a coefficient by $\bar{q} \cdot S \cdot \bar{c} \dots$ is the textbook formula, not a JSBSim convention. The corollary is that when something goes wrong, the bug is almost always in the data, the units, or the signs — not in the simulator. Trust the framework, instrument it (the property tree makes instrumentation trivial), and iterate.

The model is wrong, but the simulator is right. — folklore

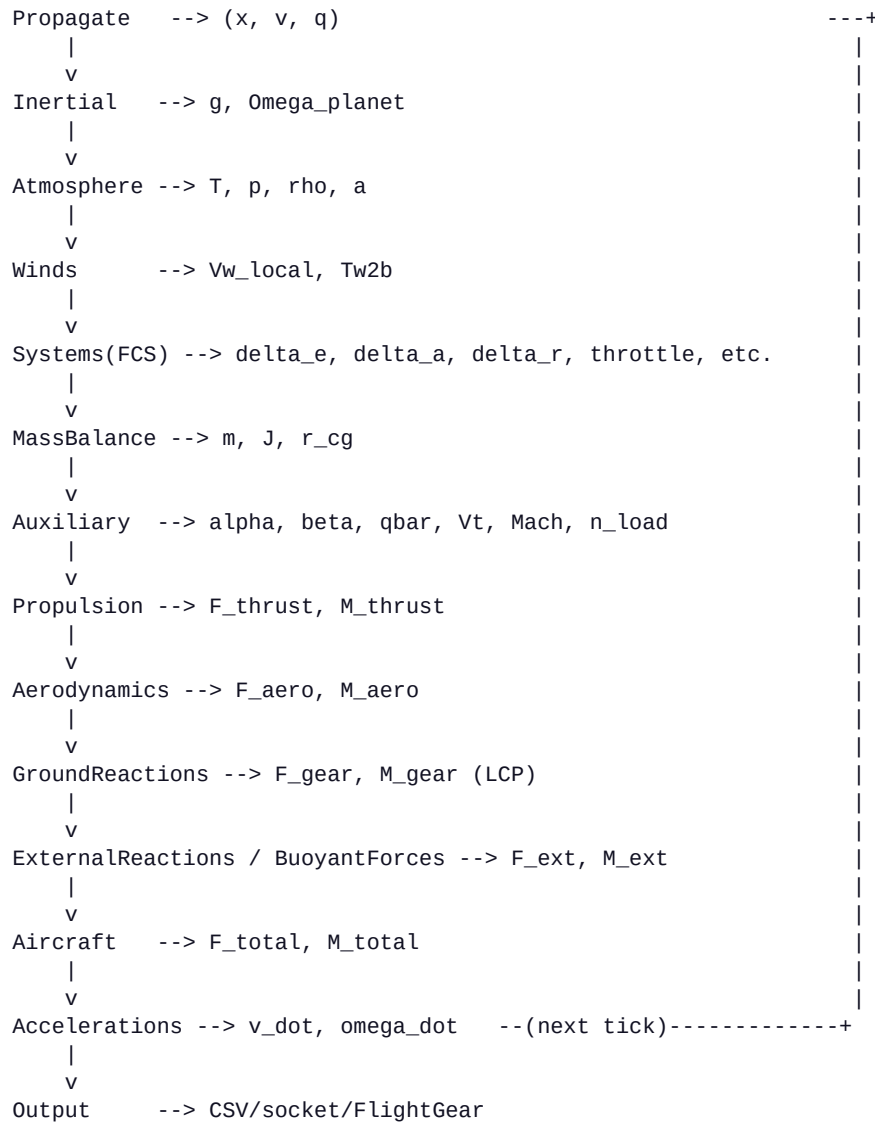
Chapter 10: Visual Diagrams of the FDM Loop

JSBSim has no built-in graphical documentation. The ASCII diagrams in this chapter are reverse-engineered from the source and faithfully reflect data flow as of v2.0.

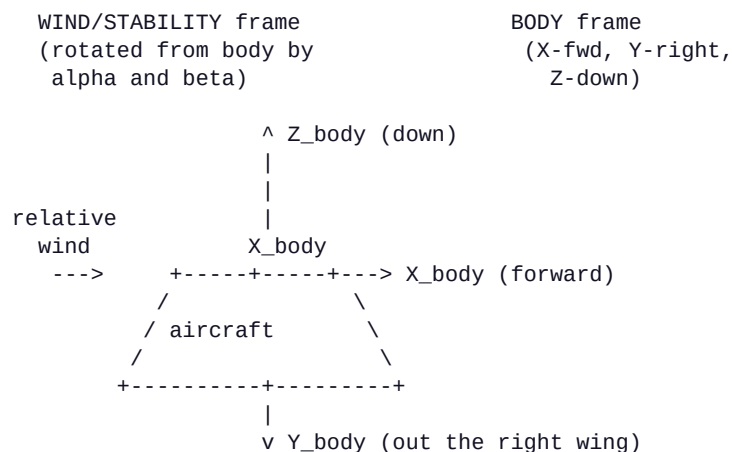
10.1 Top-level simulation loop



10.2 Per-tick data flow between models



10.3 Body-frame, wind-frame and structural-frame sketch



alpha = angle between V_{rel} and the body X axis in the X-Z plane
 beta = angle between V_{rel} and the body X-Z plane

STRUCTURAL frame (used in <location> elements of the XML):

X_struct = aft (typically),
 Y_struct = right,
 Z_struct = up.

JSBSim flips X and Z internally when converting to body frame.

10.4 Aerodynamic axis-to-force flow



Chapter 11: Propulsion — Per-Engine-Type Reference

11.1 Piston engines

FGPiston models a four-stroke piston engine with manifold-absolute-pressure dynamics, mixture and throttle controls, magneto failures and altitude derating. Configuration parameters:

```
<piston_engine name="NewEngine">
  <minmp      unit="INHG"> 10.0 </minmp>      <!-- idle MAP -->
  <maxmp      unit="INHG"> 28.5 </maxmp>      <!-- max MAP (SL) -->
  <displacement unit="IN3"> 320 </displacement>
  <maxhp>      160 </maxhp>      <!-- SL max power -->
  <bsfc>        0.45 </bsfc>      <!-- brake-spec fuel csmg -->
  <cycles>      4 </cycles>      <!-- 4 or 2-stroke -->
  <idlerpm>     600 </idlerpm>
  <maxrpm>      2700 </maxrpm>
  <maxthrottle> 1.0 </maxthrottle>
  <minthrottle> 0.05 </minthrottle>
  <sparkfaildrop> 0.1 </sparkfaildrop>
  <numboostspeeds> 1 </numboostspeeds>
  <boostoverride> 0 </boostoverride>
  <ratedpower> 160 </ratedpower>
  <ratedrpm>    2700 </ratedrpm>
  <ratedaltitude unit="FT"> 0 </ratedaltitude>
</piston_engine>
```

Power output is modelled as a parabolic relation in MAP and a linear-with-throttle/RPM scaling. Mixture affects density altitude — lean of peak penalises power, rich of peak penalises fuel flow. The model accounts for ram-air recovery at high speed.

11.2 Turbine (turbojet/turbofan)

FGTurbine implements a two-spool engine with idle/military/afterburner regions and lookup-table thrust:

```
<turbine_engine name="F100">
  <milthrust unit="LBS"> 17800 </milthrust>
  <maxthrust unit="LBS"> 29100 </maxthrust> <!-- AB rating -->
  <bypassratio> 0.35 </bypassratio>
  <tsfc>        0.74 </tsfc>      <!-- cruise SFC -->
  <atsfc>       2.05 </atsfc>      <!-- AB SFC -->
  <bleed>       0.03 </bleed>
  <idlen1>      30.0 </idlen1>
  <idlen2>      60.0 </idlen2>
  <maxn1>      100.0 </maxn1>
  <maxn2>      100.0 </maxn2>
  <augmented>   1 </augmented>    <!-- has AB -->
  <augmethod>   1 </augmethod>
  <injected>    0 </injected>

  <function name="IdleThrust">
    <table>
      <independentVar lookup="row">velocities/mach</independentVar>
      <independentVar lookup="column">atmosphere/density-altitude</independentVar>
      <tableData>
        0      20000      40000      60000
        0.0    0.0430    0.0488    0.0533    0.0593
        0.4    0.0356    0.0418    0.0466    0.0524
        0.8    0.0235    0.0312    0.0376    0.0445
        1.2    0.0070    0.0185    0.0272    0.0364
        1.6    -0.0094    0.0040    0.0156    0.0276
      </tableData>
    </table>
  </function>

  <function name="MilThrust">
```

```

        <table>...</table>          <!-- normalised thrust 0..1 -->
    </function>

    <function name="AugThrust">
        <table>...</table>          <!-- AB normalised thrust -->
    </function>
</turbine_engine>

```

Each thrust table is normalised to 1.0 at the SL static condition; the model multiplies by the relevant rating (milthrust or maxthrust). Spool-up is modelled with first-order lags; the property propulsion/engine[n]/n2-norm can be fed to FCS to gate flap deployment or thrust-reverser logic.

11.3 Turboprop

FGTurboProp couples a turbine core to a propeller. The XML adds a maxpower in shaft horsepower and a betarangeend:

```

<turboprop_engine name="PW100">
    <milthrust unit="LBS"> 0 </milthrust>
    <maxpower unit="HP"> 1200 </maxpower>
    <idlen1> 50 </idlen1>
    <maxn1> 100 </maxn1>
    <betarangeend> 0.20 </betarangeend> <!-- pitch override below this -->
    <reversemaxpower> 0.50 </reversemaxpower>
    <function name="EnginePowerVC">
        <table>...</table>
    </function>
    <function name="EnginePowerAltitude">
        <table>...</table>
    </function>
</turboprop_engine>

```

The thruster must be a <propeller> (separate XML file) and the propeller's pitch is controlled either by a governor (constant speed) or directly by FCS commands.

11.4 Rocket engines

FGRocket models a fixed-Isp rocket. Variable thrust is supported via throttle and a thrust-vs-time table; thrust vectoring is supported by a <nozzle> with steerable PYR angles. Suitable for boosters, sounding rockets, and missile models.

```

<rocket_engine name="SolidBooster">
    <isp> 265.0 </isp>          <!-- specific impulse, sec -->
    <builduptime> 0.2 </builduptime>    <!-- ignition transient -->
    <thrust_table>
        <table>                  <!-- F vs t in sec -->
            <independentVar>propulsion/engine[0]/thrust-time</independentVar>
            <tableData>
                0.0  1500
                0.5  3200
                3.0  3200
                4.0  1000
                4.5   0
            </tableData>
        </table>
    </thrust_table>
</rocket_engine>

```

11.5 Electric and brushless DC motors

```
<electric_engine name="E305">
  <power unit="WATTS"> 1000 </power>  <!-- max shaft power -->
</electric_engine>

<brushless_dc_motor name="OutrunnerBLDC">
  <maxvolts>      22.2 </maxvolts>      <!-- 6S LiPo -->
  <velocityconstant> 920 </velocityconstant>  <!-- Kv, rpm/V -->
  <coilresistance>  0.02 </coilresistance>  <!-- ohms -->
  <noloadcurrent>   1.0 </noloadcurrent>      <!-- A -->
</brushless_dc_motor>
```

Both models compute shaft torque from throttle (PWM) and a back-EMF curve. FGBrushLessDCMotor explicitly tracks current draw, which is useful for battery-life modelling — multiply current by terminal voltage for instantaneous power, then integrate against tank capacity to model battery state of charge.

11.6 Helicopter rotor

FGRotor models a helicopter main or tail rotor using an analytical blade-element approach with inflow modelling, flapping dynamics and ground-effect. It is complex — for a first pass start from the J3Cub piston piston or the X15 examples in aircraft/ rather than from scratch.

11.7 Propellers

Propellers live in engine/prop_*.xml and reference C_T and C_P tables vs advance ratio J :

```
<propeller name="prop_75in2f">
  <ixx>      1.67 </ixx>
  <diameter unit="IN"> 75.0 </diameter>
  <numblades>      2 </numblades>
  <gearratio>      1.0 </gearratio>
  <cp_factor>      1.0 </cp_factor>
  <ct_factor>      1.0 </ct_factor>
  <minpitch>      10.0 </minpitch>  <!-- variable-pitch range -->
  <maxpitch>      35.0 </maxpitch>
  <minrpm>        640 </minrpm>      <!-- governor range -->
  <maxrpm>        2700 </maxrpm>
  <constspeed>    1 </constspeed>  <!-- 1 = constant-speed prop -->
  <table name="C_THRUST" type="internal">
    <tableData>
      0.00  0.0863
      0.10  0.0837
      0.20  0.0792
      0.30  0.0727
      0.40  0.0643
      0.50  0.0540
    </tableData>
  </table>
  <table name="C_POWER" type="internal">
    <tableData> ... </tableData>
  </table>
</propeller>
```

Use p_factor on the thruster to model the slight nose-left yawing moment from prop disc asymmetry at high α . sense is +1 for clockwise (viewed from cockpit) and -1 for counter-clockwise; this matters for torque reaction direction and twin-engine "critical-engine" effects.

11.8 Fuel tanks

```
<tank type="FUEL" number="0" name="LeftWingTank">
  <location unit="IN">  <x>56</x><y>-112</y><z>59.4</z> </location>
  <drain_location unit="IN">
    <x>56</x><y>-112</y><z>50</z>          <!-- pickup point -->
  </drain_location>
  <type>AVGAS</type>          <!-- or JET-A, RP-1, ... -->
  <capacity unit="LBS"> 185 </capacity>
  <contents unit="LBS"> 100 </contents>    <!-- initial fuel -->
  <density unit="LBS/GAL"> 6.0 </density>
  <temperature> 50 </temperature>        <!-- degF -->
  <standpipe unit="LBS"> 5 </standpipe>    <!-- unusable bottom -->
  <unusable-volume unit="GAL"> 0.5 </unusable-volume>
  <priority> 1 </priority>                <!-- 1 = consume first -->
</tank>
```

Tanks contribute mass and inertia (parallel-axis-shifted from tank location to CG) that decrease as fuel is consumed. Multiple tanks with different priorities let you model the real fuel system schedule (e.g. "transfer from aux to main first").

Chapter 12: Flight Control Components — Reference

Every FCS component has the same shape: a name, one or more `<input>` properties, component-specific configuration, optional `<clipto>`, and an `<output>` property that other components downstream can read. The output is computed in the *order* in which components appear in a channel.

12.1 Summer / Pure gain / Aerosurface scale

```
<summer name="trim_sum">                                <!-- y = sum(inputs) + bias -->
  <input>fcs/elevator-cmd-norm</input>
  <input>fcs/pitch-trim-cmd-norm</input>
  <bias>0.0</bias>
  <clipto> <min>-1</min><max>1</max> </clipto>
</summer>

<pure_gain name="elev_gain">                             <!-- y = k * x -->
  <input>fcs/trim_sum</input>
  <gain>2.5</gain>                                       <!-- can be a property -->
</pure_gain>

<scheduled_gain name="yaw_damper_gain"> <!-- k = table(prop) -->
  <input>velocities/r-aero-rad_sec</input>
  <table>
    <independentVar>velocities/mach</independentVar>
    <tableData>
      0.0    0.0
      0.3    1.0
      0.9    1.2
    </tableData>
  </table>
  <output>fcs/yaw-damper-cmd</output>
</scheduled_gain>

<aerosurface_scale name="elev_to_rad">
  <input>fcs/trim_sum</input>
  <gain>0.01745</gain>                                <!-- multiply input first -->
  <domain> <min>-1</min><max>1</max> </domain>          <!-- input range -->
  <range> <min>-28</min><max>23</max> </range>          <!-- output range -->
  <output>fcs/elevator-pos-rad</output>
</aerosurface_scale>
```

12.2 Filters

Filters use Tustin (bilinear) discretisation. Each has a characteristic constant $c1..c5$ that maps to the textbook coefficients.

```
<lag_filter name="y">                                    <!-- H = c1/(s+c1) -->
  <input>fcs/x</input>
  <c1>500</c1>                                           <!-- time constant 1/500 s -->
</lag_filter>

<lead_lag_filter name="y">                               <!-- H = (c1 s + c2)/(c3 s + c4) -->
  <input>fcs/x</input>
  <c1>0.5</c1> <c2>1</c2> <c3>0.05</c3> <c4>1</c4>
</lead_lag_filter>

<washout_filter name="y">                                <!-- H = s/(s+c1) -->
  <input>velocities/q-rad_sec</input>
  <c1>1.0</c1>
</washout_filter>

<second_order_filter name="y">                           <!-- H = (c1 s^2 + c2 s + c3)/(c4 s^2 + c5 s + c6) -->
  <input>fcs/x</input>
  <c1>0</c1><c2>0</c2><c3>4</c3>
```

```
<c4>1</c4><c5>2.0</c5><c6>4</c6>      <!-- 2 Hz, zeta=0.5 -->
</second_order_filter>
```

12.3 Integrator

```
<integrator name="alpha_int">
  <input>aero/alpha-rad</input>
  <c1>1.0</c1>                                <!-- integration gain -->
  <trigger>fcs/integrator-trigger</trigger> <!-- 0=run, 1=hold, -1=reset -->
</integrator>
```

The trigger property lets you anti-windup with conventional logic: hold while saturated, reset on disengage.

12.4 PID controller

```
<pid name="AltitudeHold">
  <input>fcs/altitude-error-ft</input>
  <kp>0.05</kp>
  <ki type="ab3">0.001</ki>      <!-- ab2/ab3 = Adams-Bashforth -->
  <kd>0.10</kd>
  <trigger>ap/altitude-hold-engaged</trigger>
  <clipto> <min>-0.5</min><max>0.5</max> </clipto>
</pid>
```

12.5 Kinematic, deadband, switch

```
<kinematic name="flaps">                                <!-- discrete positions w/ traverse times -->
  <input>fcs/flap-cmd-norm</input>
  <traverse>
    <setting><position> 0</position><time>0</time></setting>
    <setting><position>10</position><time>2</time></setting>
    <setting><position>30</position><time>4</time></setting>
  </traverse>
  <output>fcs/flap-pos-deg</output>
</kinematic>

<deadband name="stick_db">                                <!-- zero out small inputs -->
  <input>fcs/elevator-cmd-norm</input>
  <width>0.02</width>
</deadband>

<switch name="gear_switch">                                <!-- conditional output -->
  <default value="0"/>
  <test logic="AND" value="1">
    gear/gear-cmd-norm ge 0.5
    velocities/vc-kts le 250
  </test>
  <output>fcs/gear-extend</output>
</switch>
```

12.6 Sensor and actuator wrapping

Use <sensor> to simulate imperfect measurements (lag, bias, noise) and <actuator> to simulate imperfect surface actuation (rate limit, hysteresis, deadband, fail modes). Both are useful for fault-injection studies.

```
<sensor name="ias_sensor">
  <input>velocities/vc-kts</input>
  <lag>0.5</lag>      <!-- first-order lag tau -->
  <bias>2</bias>      <!-- additive offset -->
  <drift_rate>0.01</drift_rate> <!-- units/sec -->
  <gain>1.005</gain>
  <noise variation="PERCENT">0.5</noise>
  <quantization name="adc12bit">
    <bits>12</bits><min>0</min><max>500</max>
  </quantization>
  <delay>0.1</delay>      <!-- pure time delay -->
  <output>instr/ias-indicated</output>
```

```
</sensor>

<actuator name="elev_act">
  <input>fcs/elevator-cmd-rad</input>
  <lag>0.05</lag>
  <rate_limit sense="incr">0.6</rate_limit>
  <rate_limit sense="decr">0.6</rate_limit>
  <bias>0</bias>
  <deadband_width>0.0017</deadband_width>
  <hysteresis_width>0.0035</hysteresis_width>
  <fail_zero> fcs/elev-act-fail-zero </fail_zero>
  <fail_hardover> fcs/elev-act-fail-hard </fail_hardover>
  <fail_stuck> fcs/elev-act-fail-stuck </fail_stuck>
  <output>fcs/elevator-pos-rad</output>
</actuator>
```

Chapter 13: Atmosphere, Winds, and Earth

13.1 ISA 1976 standard atmosphere

FGStandardAtmosphere evaluates the 1976 US Standard Atmosphere — eight piecewise lapse-rate segments from sea level to 86 km geometric altitude. The model exposes:

atmosphere/T-R	static temperature, Rankine
atmosphere/T-sl-R	sea-level reference temperature
atmosphere/T-dev-R	local deviation from standard
atmosphere/P-psf	static pressure, lb/ft ²
atmosphere/P-sl-psf	
atmosphere/rho-slugs_ft3	density
atmosphere/a-fps	speed of sound
atmosphere/density-altitude	
atmosphere/pressure-altitude	

Custom atmosphere. Inherit from FGAtmosphere in C++ and override Calculate(double altitudeASL). For most needs the standard model plus a temperature offset (atmosphere/delta-T) is enough — shifting T while keeping P constant gives a density-altitude shift, which is how density-altitude effects (hot-and-high) are usually modelled.

13.2 Winds and shear

atmosphere/wind-north-fps	steady wind from FGWinds
atmosphere/wind-east-fps	(NED frame, points TO direction wind blows toward)
atmosphere/wind-down-fps	
atmosphere/wind-mag-fps	magnitude
atmosphere/psiw-rad	wind heading
atmosphere/total-wind-north-fps	steady + turbulence + gust + shear
atmosphere/turb-rate-rad_sec	Dryden/Karman turbulence angular component

Turbulence models: select via atmosphere/turb-type:

0 ttNone	no turbulence
1 ttStandard	legacy white-noise model
2 ttCulp	John Culp's model (used in FlightGear)
3 ttMilspec	MIL-F-8785C Dryden
4 ttTustin	MIL-F-8785C Tustin discrete-Dryden

For the MIL-spec models you set atmosphere/turbulence/milspec/severity on a 0-7 scale; the model auto-selects scale lengths and intensities according to the spec.

13.3 Gusts (discrete)

A discrete "1-cosine" gust can be triggered by setting:

```

atmosphere/cosine-gust-start -> set to 1 to start
atmosphere/cosine-gust-frame -> 1 BODY, 2 WIND, 3 LOCAL
atmosphere/cosine-gust-duration
atmosphere/cosine-gust-magnitude-ft_sec
atmosphere/cosine-gust-startup-duration
atmosphere/cosine-gust-steady-duration
atmosphere/cosine-gust-end-duration
atmosphere/cosine-gust-X-velocity (and Y, Z in chosen frame)

```

13.4 Microburst

A three-component microburst model (Vicroy + Mulgund) is available via atmosphere/turbulence-cosine-set. Used for go-around and wind-shear-recovery training scenarios.

13.5 Inertial: gravity and Earth rotation

FGInertial publishes the gravity vector and the planet rotation rate. Two gravity models are available, selected via simulation/gravity-model:

- 0 Spherical inverse-square gravity (uniform mass distribution)
- 1 WGS-84 ellipsoid with J2 term (default)

simulation/gravitational-torque -> 0/1 (default 0)
 Adds tidal torque on extended bodies; relevant for spacecraft.

Planet rotation: 7.2921151467e-5 rad/s about ECI Z (sidereal day).

WGS-84 ellipsoid:

semi-major axis a	= 6378137.0 m
semi-minor axis b	= 6356752.3142 m
flattening f	= 1/298.257223563
GM	= 3.986004418e14 m ³ /s ²
J2	= 1.08263e-3

Why this matters even for low-altitude flight. The Coriolis acceleration at Mach 0.8 cruise is on the order of 0.003 m/s² — small but non-zero. JSBSim correctly accounts for it, which is part of why it passes the NASA 2015 check cases. If you compare against simulators that use a flat-Earth approximation, expect small steady-state heading drifts when flying long distances east-west.

Chapter 14: Quaternion Attitude — A Primer for the Code

Reading FGPropagate and FGQuaternion is much easier if you already speak quaternions. This chapter is a quick refresher with JSBSim-specific notation.

14.1 Definition and conventions

A unit quaternion is a 4-tuple $q = (q_0, q_1, q_2, q_3)$ with norm 1. JSBSim follows the Hamilton ($i^2 = j^2 = k^2 = ijk = -1$) convention; FGQuaternion stores them in the order (w, x, y, z) — i.e. the scalar component first.

$$q = \cos(\theta/2) + \hat{n} \cdot \sin(\theta/2)$$

where $\hat{n}$ is the rotation axis and θ is the rotation angle. The quaternion that rotates a vector in the inertial frame into the body frame is $q_{i \rightarrow b}$ and the equivalent passive rotation is

$$v_{\text{body}} = q^* \cdot v_{\text{inertial}} \cdot q$$

(with vectors lifted to pure quaternions, and $\cdot$ denoting Hamilton product). This is exactly what FGQuaternion::GetTransformationMatrix() precomputes as a 3×3 rotation matrix to use for fast frame transforms.

14.2 Kinematic equation

$$\dot{q} = \frac{1}{2} \cdot q \cdot \omega_{b/i}$$

Here $\omega_{b/i}$ is the inertial angular rate expressed as a pure quaternion $(0, p, q, r)$. FGQuaternion::GetQDot() implements this in 12 floating-point operations.

```
FGQuaternion FGQuaternion::GetQDot(const FGColumnVector3& PQR) const {
    return FGQuaternion(
        -0.5*( data[1]*PQR(eP) + data[2]*PQR(eQ) + data[3]*PQR(eR)),
        0.5*( data[0]*PQR(eP) - data[3]*PQR(eQ) + data[2]*PQR(eR)),
        0.5*( data[3]*PQR(eP) + data[0]*PQR(eQ) - data[1]*PQR(eR)),
        0.5*(-data[2]*PQR(eP) + data[1]*PQR(eQ) + data[0]*PQR(eR))
    );
}
```

14.3 Renormalisation

Numerical integration of $\dot{q}$ with Euler or Adams-Bashforth schemes does not preserve the unit norm — over time the quaternion drifts off the unit 3-sphere, equivalent to introducing a spurious dilation. JSBSim renormalises every tick by dividing by the magnitude (FGQuaternion::Normalize()). The Buss integrators avoid this by integrating with the exponential map directly:

$$q(t+\Delta t) = q(t) \cdot \exp(\frac{1}{2} \cdot \Delta t \cdot \omega)$$

which is exact for constant ω and stays unit-norm to floating-point precision.

14.4 Euler angle extraction

FGQuaternion::GetEulerDeg() returns the conventional aircraft Euler angles (ϕ, θ, ψ) in degrees using a 3-2-1 (Z-Y-X intrinsic) sequence. The singularity at $\theta = \pm 90^\circ$ is unavoidable — quaternions exist precisely to *avoid* the singularity in the integration, but they still have to project through it when you read out Euler angles. For highly aerobatic aircraft, do all your work in quaternion space or in DCM space and only extract Euler at the end.

Chapter 15: Complete Worked Example: A Simple Aircraft

This chapter walks every section of a minimal but flyable aircraft — an Acme A-1 light single — pulling values from first-principles geometry, a piston engine reused from the library, and CFD-derived aerodynamic coefficients. The complete XML is reproduced in pieces with commentary.

15.1 Header and reference data

```
<?xml version="1.0"?>
<fdm_config name="AcmeA1" version="2.0" release="ALPHA">

<fileheader>
  <author>Engineering Team</author>
  <filecreationdate>2026-05-24</filecreationdate>
  <description>Acme A-1, single-engine 2-seat sport aircraft.</description>
</fileheader>

<metrics>
  <wingarea unit="FT2"> 140 </wingarea>      <!-- S -->
  <wingspan unit="FT"> 32 </wingspan>         <!-- b -->
  <chord unit="FT"> 4.4 </chord>              <!-- cbar -->
  <htailarea unit="FT2"> 18 </htailarea>
  <htailarm unit="FT"> 14 </htailarm>
  <vtailarea unit="FT2"> 15 </vtailarea>
  <vtailarm unit="FT"> 14 </vtailarm>
  <location name="AERORP" unit="IN">
    <x>40</x><y>0</y><z>30</z>                <!-- 25% MAC -->
  </location>
  <location name="EYEPOINT" unit="IN">
    <x>30</x><y>-10</y><z>50</z>
  </location>
  <location name="VRP" unit="IN">
    <x>0</x><y>0</y><z>0</z>
  </location>
</metrics>
```

15.2 Mass, balance and inertia

```
<mass_balance>
  <!-- Inertias from CAD about the empty-weight CG, structural frame -->
  <ixx unit="SLUG*FT2"> 800 </ixx>
  <iyy unit="SLUG*FT2"> 1100 </iyy>
  <izz unit="SLUG*FT2"> 1700 </izz>
  <ixz unit="SLUG*FT2"> 0 </ixz>
  <emptywt unit="LBS"> 1250 </emptywt>
  <location name="CG" unit="IN">
    <x>39</x><y>0</y><z>30</z>                <!-- ~24% MAC -->
  </location>
  <pointmass name="Pilot">
    <weight unit="LBS">175</weight>
    <location unit="IN"><x>34</x><y>-12</y><z>32</z></location>
  </pointmass>
  <pointmass name="Passenger">
    <weight unit="LBS">175</weight>
    <location unit="IN"><x>34</x><y> 12</y><z>32</z></location>
  </pointmass>
  <pointmass name="Baggage">
    <weight unit="LBS"> 60</weight>
    <location unit="IN"><x>70</x><y>0</y><z>30</z></location>
  </pointmass>
</mass_balance>
```


15.3 Landing gear

```

<ground_reactions>
  <contact type="BOGEY" name="NOSE">
    <location unit="IN"><x>-10</x><y>0</y><z>-20</z></location>
    <static_friction>0.8</static_friction>
    <dynamic_friction>0.5</dynamic_friction>
    <rolling_friction>0.02</rolling_friction>
    <spring_coeff unit="LBS/FT"> 1500 </spring_coeff>
    <damping_coeff unit="LBS/FT/SEC"> 500 </damping_coeff>
    <max_steer unit="DEG">15</max_steer>
    <brake_group>NONE</brake_group>
  </contact>
  <contact type="BOGEY" name="LEFT_MAIN">
    <location unit="IN"><x>55</x><y>-40</y><z>-18</z></location>
    <static_friction>0.8</static_friction>
    <dynamic_friction>0.5</dynamic_friction>
    <rolling_friction>0.02</rolling_friction>
    <spring_coeff unit="LBS/FT"> 4500 </spring_coeff>
    <damping_coeff unit="LBS/FT/SEC">1300 </damping_coeff>
    <brake_group>LEFT</brake_group>
  </contact>
  <contact type="BOGEY" name="RIGHT_MAIN">
    <location unit="IN"><x>55</x><y> 40</y><z>-18</z></location>
    <static_friction>0.8</static_friction>
    <dynamic_friction>0.5</dynamic_friction>
    <rolling_friction>0.02</rolling_friction>
    <spring_coeff unit="LBS/FT"> 4500 </spring_coeff>
    <damping_coeff unit="LBS/FT/SEC">1300 </damping_coeff>
    <brake_group>RIGHT</brake_group>
  </contact>
  <contact type="STRUCTURE" name="LEFT_TIP">
    <location unit="IN"><x>40</x><y>-192</y><z>30</z></location>
    <static_friction>0.2</static_friction>
    <dynamic_friction>0.2</dynamic_friction>
    <spring_coeff unit="LBS/FT"> 20000 </spring_coeff>
    <damping_coeff unit="LBS/FT/SEC"> 2000 </damping_coeff>
  </contact>
  <contact type="STRUCTURE" name="RIGHT_TIP">
    <location unit="IN"><x>40</x><y> 192</y><z>30</z></location>
    <static_friction>0.2</static_friction>
    <dynamic_friction>0.2</dynamic_friction>
    <spring_coeff unit="LBS/FT"> 20000 </spring_coeff>
    <damping_coeff unit="LBS/FT/SEC"> 2000 </damping_coeff>
  </contact>
</ground_reactions>

```

15.4 Propulsion

```
<propulsion>
  <engine file="eng_io360">          <!-- 180 hp 4-cyl -->
    <feed>0</feed>
    <feed>1</feed>
    <thruster file="prop_77in">
      <location unit="IN"><x>-30</x><y>0</y><z>30</z></location>
      <orient unit="DEG"><pitch>2</pitch></orient>
      <sense>1</sense>
      <p_factor>5</p_factor>
    </thruster>
  </engine>
  <tank type="FUEL" number="0">
    <location unit="IN"><x>40</x><y>-80</y><z>32</z></location>
    <capacity unit="LBS">160</capacity>
    <contents unit="LBS">120</contents>
    <type>AVGAS</type>
  </tank>
  <tank type="FUEL" number="1">
    <location unit="IN"><x>40</x><y> 80</y><z>32</z></location>
    <capacity unit="LBS">160</capacity>
    <contents unit="LBS">120</contents>
    <type>AVGAS</type>
  </tank>
</propulsion>
```

15.5 Flight control system

```

<flight_control name="FCS: AcmeA1">
  <channel name="Pitch">
    <summer name="pitch_sum">
      <input>fcs/elevator-cmd-norm</input>
      <input>fcs/pitch-trim-cmd-norm</input>
      <clipto><min>-1</min><max>1</max></clipto>
    </summer>
    <aerosurface_scale name="elevator">
      <input>fcs/pitch_sum</input>
      <range><min>-25</min><max>20</max></range>
      <gain>0.01745</gain>
      <output>fcs/elevator-pos-rad</output>
    </aerosurface_scale>
  </channel>

  <channel name="Roll">
    <summer name="roll_sum">
      <input>fcs/aileron-cmd-norm</input>
      <input>fcs/roll-trim-cmd-norm</input>
      <clipto><min>-1</min><max>1</max></clipto>
    </summer>
    <aerosurface_scale name="left_aileron">
      <input>fcs/roll_sum</input>
      <range><min>-20</min><max>15</max></range>
      <gain>0.01745</gain>
      <output>fcs/left-aileron-pos-rad</output>
    </aerosurface_scale>
    <aerosurface_scale name="right_aileron">
      <input>fcs/roll_sum</input>
      <range><min>-15</min><max>20</max></range>
      <gain>-0.01745</gain>
      <output>fcs/right-aileron-pos-rad</output>
    </aerosurface_scale>
  </channel>

  <channel name="Yaw">
    <summer name="yaw_sum">
      <input>fcs/rudder-cmd-norm</input>
      <input>fcs/yaw-trim-cmd-norm</input>
      <clipto><min>-1</min><max>1</max></clipto>
    </summer>
    <aerosurface_scale name="rudder">
      <input>fcs/yaw_sum</input>
      <range><min>-20</min><max>20</max></range>
      <gain>0.01745</gain>
      <output>fcs/rudder-pos-rad</output>
    </aerosurface_scale>
  </channel>

  <channel name="Flaps">
    <kinematic name="flaps">
      <input>fcs/flap-cmd-norm</input>
      <traverse>
        <setting><position> 0</position><time>0</time></setting>
        <setting><position>10</position><time>3</time></setting>
        <setting><position>20</position><time>2</time></setting>
        <setting><position>30</position><time>2</time></setting>
      </traverse>
      <output>fcs/flap-pos-deg</output>
    </kinematic>
  </channel>
</flight_control>

```

15.6 Aerodynamics (representative values from CFD)

```

<aerodynamics>
  <alphalimits unit="RAD">
    <min>-0.087</min><max>0.30</max>
  </alphalimits>
  <hysteresis_limits unit="RAD">
    <min>0.20</min><max>0.30</max>
  </hysteresis_limits>

  <!-- ===== LIFT ===== -->
  <axis name="LIFT">
    <function name="aero/coefficient/CLwbh">
      <description>Wing-body-tail lift vs alpha (clean)</description>
      <product>
        <property>aero/qbar-psf</property>
        <property>metrics/Sw-sqft</property>
        <table>
          <independentVar>aero/alpha-rad</independentVar>
          <tableData>
            -0.087  -0.250
            -0.044   0.100
            0.000   0.300
            0.087   0.760
            0.175   1.230
            0.262   1.500
            0.300   1.200    <!-- post-stall -->
          </tableData>
        </table>
      </product>
    </function>

    <function name="aero/coefficient/CLDf">
      <description>Lift increment due to flap</description>
      <product>
        <property>aero/qbar-psf</property>
        <property>metrics/Sw-sqft</property>
        <table>
          <independentVar>fcs/flap-pos-deg</independentVar>
          <tableData>
            0    0.00
            10   0.20
            20   0.35
            30   0.45
          </tableData>
        </table>
      </product>
    </function>

    <function name="aero/coefficient/CLde">
      <description>Lift due to elevator</description>
      <product>
        <property>aero/qbar-psf</property>
        <property>metrics/Sw-sqft</property>
        <property>fcs/elevator-pos-rad</property>
        <value>0.43</value>
      </product>
    </function>
  </axis>

  <!-- ===== DRAG ===== -->
  <axis name="DRAG">
    <function name="aero/coefficient/CD0">
      <product>
        <property>aero/qbar-psf</property>
        <property>metrics/Sw-sqft</property>
        <value>0.025</value>
      </product>
    </function>
    <function name="aero/coefficient/CDi">

```

```

        <description>Induced drag,  $CL^2 / (\pi * AR * e)$ </description>
        <product>
            <property>aero/qbar-psf</property>
            <property>metrics/Sw-sqft</property>
            <quotient>
                <property>aero/cl-squared</property>
                <value>20.4</value>          <!-- pi * AR * e ~ 20 -->
            </quotient>
        </product>
    </function>
</axis>

<!-- ===== PITCH ===== -->
<axis name="PITCH">
    <function name="aero/coefficient/Cm0">
        <product>
            <property>aero/qbar-psf</property>
            <property>metrics/Sw-sqft</property>
            <property>metrics/cbarw-ft</property>
            <value>-0.04</value>
        </product>
    </function>
    <function name="aero/coefficient/Cma">
        <description>Pitch moment slope (must be negative)</description>
        <product>
            <property>aero/qbar-psf</property>
            <property>metrics/Sw-sqft</property>
            <property>metrics/cbarw-ft</property>
            <property>aero/alpha-rad</property>
            <value>-0.5</value>
        </product>
    </function>
    <function name="aero/coefficient/Cmde">
        <product>
            <property>aero/qbar-psf</property>
            <property>metrics/Sw-sqft</property>
            <property>metrics/cbarw-ft</property>
            <property>fcs/elevator-pos-rad</property>
            <value>-1.10</value>
        </product>
    </function>
    <function name="aero/coefficient/Cmq">
        <description>Pitch damping</description>
        <product>
            <property>aero/qbar-psf</property>
            <property>metrics/Sw-sqft</property>
            <property>metrics/cbarw-ft</property>
            <property>aero/ci2vel</property>
            <property>velocities/q-aero-rad_sec</property>
            <value>-12.0</value>
        </product>
    </function>
</axis>

<!-- ===== SIDE ===== -->
<axis name="SIDE">
    <function name="aero/coefficient/CYb">
        <product>
            <property>aero/qbar-psf</property>
            <property>metrics/Sw-sqft</property>
            <property>aero/beta-rad</property>
            <value>-0.7</value>
        </product>
    </function>
</axis>

<!-- ===== ROLL ===== -->
<axis name="ROLL">
    <function name="aero/coefficient/Clb">
        <description>Dihedral effect (must be negative)</description>
        <product>

```

```

        <property>aero/qbar-psf</property>
        <property>metrics/Sw-sqft</property>
        <property>metrics/bw-ft</property>
        <property>aero/beta-rad</property>
        <value>-0.08</value>
    </product>
</function>
<function name="aero/coefficient/Clp">
    <description>Roll damping</description>
    <product>
        <property>aero/qbar-psf</property>
        <property>metrics/Sw-sqft</property>
        <property>metrics/bw-ft</property>
        <property>aero/bi2vel</property>
        <property>velocities/p-aero-rad_sec</property>
        <value>-0.46</value>
    </product>
</function>
<function name="aero/coefficient/Clda">
    <product>
        <property>aero/qbar-psf</property>
        <property>metrics/Sw-sqft</property>
        <property>metrics/bw-ft</property>
        <property>fcs/left-aileron-pos-rad</property>
        <value>0.22</value>
    </product>
</function>
</axis>

<!-- ===== YAW ===== -->
<axis name="YAW">
    <function name="aero/coefficient/Cnb">
        <description>Weathercock stability (must be positive)</description>
        <product>
            <property>aero/qbar-psf</property>
            <property>metrics/Sw-sqft</property>
            <property>metrics/bw-ft</property>
            <property>aero/beta-rad</property>
            <value>0.06</value>
        </product>
    </function>
    <function name="aero/coefficient/Cnr">
        <description>Yaw damping</description>
        <product>
            <property>aero/qbar-psf</property>
            <property>metrics/Sw-sqft</property>
            <property>metrics/bw-ft</property>
            <property>aero/bi2vel</property>
            <property>velocities/r-aero-rad_sec</property>
            <value>-0.10</value>
        </product>
    </function>
    <function name="aero/coefficient/Cndr">
        <product>
            <property>aero/qbar-psf</property>
            <property>metrics/Sw-sqft</property>
            <property>metrics/bw-ft</property>
            <property>fcs/rudder-pos-rad</property>
            <value>-0.05</value>
        </product>
    </function>
</axis>
</aerodynamics>

</fdm_config>

```

Sanity checks for this model:

- $C_{L\alpha} \approx 5.4/\text{rad}$ (slope of the lift table near $\alpha=0$). Typical for a Hershey-bar wing at $AR \approx 7.3$ is around $4.7/\text{rad}$ — our number is close enough.

- $C_{L_{\max}} \approx 1.5$. Reasonable for a clean wing.
- $C_{D0} = 0.025$. Around the right number for a fixed-gear single.
- Statically stable: $C_{m\alpha} = -0.5 < 0$, $C_{n\beta} = +0.06 > 0$, $C_{l\beta} = -0.08 < 0$.
- Elevator-derivative sign: $C_{m\delta_e} = -1.1$. Trailing-edge down (positive δ_e) produces nose-down pitch — that matches our sign convention.

Chapter 16: Initial Conditions, Scripts, and Resets

16.1 Initial conditions (reset files)

The IC file sets every state variable to a self-consistent starting point for the simulation. The minimal set is position (3), attitude (3), velocity (3) — JSBSim then computes α , β , $\bar{q}$, Mach internally. You can over-specify and let JSBSim resolve, but explicit is better.

```
<?xml version="1.0"?>
<initialize name="cruise_5000">
  <!-- Position -->
  <latitude unit="DEG"> 37.6 </latitude>
  <longitude unit="DEG">-122.4 </longitude>
  <altitude unit="FT"> 5000 </altitude>           <!-- above MSL -->

  <!-- Attitude (Euler 3-2-1) -->
  <phi unit="DEG"> 0 </phi>                       <!-- bank -->
  <theta unit="DEG"> 2 </theta>                   <!-- pitch -->
  <psi unit="DEG"> 90 </psi>                      <!-- heading -->

  <!-- Pick ONE velocity spec: -->
  <ubody unit="FT/SEC"> 150 </ubody>
  <vbody unit="FT/SEC"> 0 </vbody>
  <wbody unit="FT/SEC"> 5 </wbody>
  <!-- or: <vt unit="KTS">100</vt> with <alpha unit="DEG">2</alpha> -->
  <!-- or: <vc unit="KTS">95</vc> (calibrated airspeed) -->
  <!-- or: <mach>0.55</mach> -->
</initialize>
```

Other useful IC elements:

```
<gamma unit="DEG"> 0 </gamma>           flight path angle
<vnorth>0</vnorth><veast>0</veast><vdown>0</vdown> NED wind-rel velocity
<roc unit="FT/MIN"> 0 </roc>           rate of climb
<altitudeAGL unit="FT"> 0 </altitudeAGL> above ground level
<targetNlf> 1 </targetNlf>           target load factor for trim-pullup
```

16.2 Scripts (event-based scenarios)

A script is the closed-form description of a flight test scenario: a sequence of events with conditions and property actions. The Run loop ticks the model at the specified dt until it reaches end.

```
<?xml version="1.0"?>
<runscript name="Cruise climb">
  <description>5000 ft cruise -> trim -> climb to 8000</description>
  <use aircraft="AcmeA1" initialize="cruise_5000"/>

  <run start="0" end="300" dt="0.00833333">
    <event name="Trim">
      <condition>simulation/sim-time-sec ge 0</condition>
      <set name="simulation/do_simple_trim" value="1"/>
      <notify/>
    </event>

    <event name="Throttle up">
      <condition>simulation/sim-time-sec ge 30</condition>
      <set name="fcs/throttle-cmd-norm"
        action="FG_EXP" tc="5.0" value="1.0"/>
      <notify/>
    </event>

    <event name="Climb to 8000">
      <condition>simulation/sim-time-sec ge 60</condition>
      <set name="fcs/elevator-cmd-norm"
        action="FG_RAMP" tc="3.0" value="-0.1"/>
    </event>
```



```

        <notify/>
    </event>

    <event name="Level off">
        <condition>position/h-sl-ft ge 8000</condition>
        <set name="fcs/elevator-cmd-norm" value="0"/>
        <notify>
            <property>position/h-sl-ft</property>
            <property>velocities/vc-kts</property>
        </notify>
    </event>
</run>
</runscript>

```

Action types on <set>:

FG_VALUE	set property to value immediately (default)
FG_DELTA	add value to current property
FG_RAMP	linear ramp over tc seconds
FG_EXP	first-order exponential approach with tc seconds

Conditions: any boolean expression over properties, using `ge` `le` `gt` `lt` `eq` `neq` and `or` `not`. Conditions can be nested via `<condition logic="AND|OR">`.

Chapter 17: Validation Workflow

A model that compiles and trims is only the first step. The industry-standard validation steps below catch the vast majority of modelling bugs before they reach a customer.

17.1 Step 1: trim diagnostics

Run `simulation/do_simple_trim = 1` (longitudinal) and check the resulting elevator and throttle positions are inside the actual control envelope. If trim α is near the top of the alpha-limits, the trim is unstalled by luck only; lower the cruise speed or check your $C_L(\alpha)$ curve.

17.2 Step 2: open-loop step responses

Run pitch doublets, aileron pulses and rudder kicks from a trimmed condition. Plot $p, q, r, \alpha, \beta, \phi, \theta, \psi$:

- **Phugoid**: oscillatory at ~30-60 s period, lightly damped ($\zeta \approx 0.05$). Visible as long-period altitude and airspeed swings after a pitch perturbation.
- **Short period**: ~2-5 s period, well damped ($\zeta \approx 0.5$). Visible as a rapid pitch-rate oscillation after a doublet.
- **Roll mode**: first-order, time constant ~0.5-1.5 s. Visible as exponential decay of p after an aileron kick.
- **Dutch roll**: ~3-6 s period, $\zeta \approx 0.05$ -0.3. Coupled yaw-roll after a rudder kick.
- **Spiral mode**: very slow (~30-300 s time constant), often slightly unstable for conventional aircraft.

17.3 Step 3: linear model extraction

JSBSim ships a Python utility (`python/JSBSim/utils/linearize.py`) that builds an A, B, C, D linear model at a trim point. Compute the eigenvalues of A and compare poles against the closed-form estimates of Etkin or Stevens-Lewis. The order of magnitude should agree; if a pole is in the wrong half-plane, you have a sign error in your aero coefficients.

17.4 Step 4: performance envelope check

Quantity	How to obtain in JSBSim	Compare against
V_{S0} stall speed (clean)	Trim at min α holding altitude, decrement throttle	POH or design target
V_{S1} stall speed (flaps)	Same but with flap pos = max	POH
V_{max}	Full throttle level flight, record speed	POH
Best rate of climb (V_y)	Climb at various speeds, find max ROC	POH chart
Service ceiling	Step-altitude trims, find h at ROC = 100 fpm	POH
Range	Cruise at best-economy, integrate fuel burn	POH

17.5 Step 5: handling-qualities check

If you care about pilot feel, evaluate against MIL-F-8785C / MIL-HDBK-1797 handling-qualities specifications. The most valuable single number is the C^* *parameter* (pitch-rate weighted load factor) plotted on a Cooper-Harper boundary chart at the cruise condition. JSBSim outputs all the necessary properties to compute it offline.

17.6 Step 6: flight-test comparison

Where flight-test data exists, run the same manoeuvre in JSBSim with the same inputs and overlay. The disagreements you see are a recipe for the next iteration of CFD or coefficient tuning. Beware of "chasing the flight": tuning JSBSim to match one manoeuvre often degrades others. Always validate against a held-out manoeuvre set.

Chapter 18: Common Bugs and How to Diagnose Them

These are the failure modes most newcomers encounter, in rough order of frequency. The solution column points at the first thing to inspect.

Symptom	Likely cause	First check
Aircraft "falls through" the runway at IC	Gear Z coordinate has wrong sign or wrong unit	<contact> Z negative if gear is below the structural-frame origin
Aircraft is upside down or rotates 180° on startup	ψ vs heading conventions or negated_crossproduct_inertia mismatch	Swap inertia sign attribute
Trim fails immediately	α range in CFD tables doesn't span trim α	Set <output> for α and run unwise — see what α it asks for
Trim converges but aircraft pitches down nose-first	$C_{m\alpha}$ sign error (model is statically unstable)	Plot C_m vs α ; slope must be negative
Aircraft yaws sideways uncontrollably	$C_{n\beta}$ sign error	Slope must be positive
Slow but steady roll-off	Asymmetric pointmass or aileron not centred	Check pointmass Y locations sum to zero
Pitch oscillates at 0.5-2 Hz	Insufficient C_{mq} or wrong sign	Should be negative; magnitude 5-15 for GA
Aileron input has no roll response	$C_{l\delta a}$ wrong sign or aileron pos property not matching aero coefficient property	Trace fcs/left-aileron-pos-rad through to the function
Drag enormous at trim	Forgot to subtract reference drag, or CFD coefficients in body frame fed to LIFT/DRAG axes	Compute $D = C_D \cdot \bar{q} \cdot S$ by hand; compare to $F_{x,aero,body}$
Engine produces no thrust	Tank capacity = 0 or feed indices wrong	Check propulsion/tank[n]/contents-lbs
Simulation crashes with NaN	Divide-by-zero in a function (often /Vt at zero airspeed) or quaternion gone non-unit	Add <clipto> or guard with <ifthen>
Aircraft "sinks" through ground after touch-down	Spring constant too soft for the weight	$k \geq W / 0.5 \text{ ft}$ for normal damping rates
Bounces violently on landing	Damping coefficient too low	Aim for $\zeta \approx 0.4-0.6$ ($c = 2\zeta\sqrt{km}$)
FCS output stuck at clipto	Saturated; integrator wound up	Add an anti-windup trigger to the PID
FlightGear shows the aircraft jittering	Output rate mismatch or FG/FDM timestep diverge	Set output rate to 60 Hz, FG to 60 Hz too

Chapter 19: Python API

JSBSim ships a Python module (`jsbsim`) on PyPI and conda-forge that wraps the C++ library. It is the recommended way to drive JSBSim from machine-learning code, batch parameter studies, or unit tests.

19.1 Quick start

```
import jsbsim

fdm = jsbsim.FGFDMEExec(root_dir=None) # auto-detect aircraft path
fdm.set_debug_level(0)
fdm.load_model('c172p')
fdm.load_ic('reset00', useStoredPath=True)
fdm.run_ic() # apply initial conditions

# Trim longitudinally
fdm['simulation/do_simple_trim'] = 1

# Step the model 10 seconds at 120 Hz
for _ in range(1200):
    fdm.run()

print("alt:", fdm['position/h-sl-ft'])
print("V :", fdm['velocities/vt-fps'])
```

19.2 Reading and writing properties

```
# All properties accessible by string path
fdm['fcs/elevator-cmd-norm'] = -0.1 # nose-up command
fdm['fcs/throttle-cmd-norm'] = 0.8
alpha = fdm['aero/alpha-deg']
p, q, r = (fdm[f'velocities/{ax}-rad-sec'] for ax in 'pqr')

# Snapshot the full property tree (slow)
snapshot = fdm.query_property_catalog('')
```

19.3 Trim and linearisation

```
from jsbsim.utils.linearize import linearize

fdm.load_model('c172p')
fdm.load_ic('cruise', useStoredPath=True)
fdm.run_ic()
fdm['simulation/do_simple_trim'] = 1

A, B, C, D, x_trim, u_trim = linearize(fdm)
import numpy as np
eigvals = np.linalg.eigvals(A)
print(sorted(eigvals, key=lambda e: e.real))
```

19.4 Batch parameter studies

```
def sweep_alpha(cg_x_in):
    fdm = jsbsim.FGFDMEExec(None)
    fdm.load_model('AcmeA1')
    fdm.load_ic('cruise_5000', useStoredPath=True)
    fdm['inertia/pointmass-location-X-inches[0]'] = cg_x_in
    fdm.run_ic()
    fdm['simulation/do_simple_trim'] = 1
    return fdm['aero/alpha-deg'], fdm['fcs/elevator-pos-deg']

for cg in range(30, 50, 2):
    a, e = sweep_alpha(cg)
    print(f'CG={cg} in alpha={a:.2f} elev={e:.2f}')
```

Chapter 20: Extended Property Reference

The complete property tree is published by JSBSim itself with `fdm.query_property_catalog('')`. The most useful properties for instrumentation and machine learning are grouped here.

20.1 Time

<code>simulation/sim-time-sec</code>	simulation time since reset
<code>simulation/frame</code>	tick counter
<code>simulation/dt</code>	timestep size (s)
<code>simulation/integrator/rate/rotational</code>	
<code>simulation/integrator/rate/translational</code>	
<code>simulation/integrator/position/rotational</code>	
<code>simulation/integrator/position/translational</code>	

20.2 Atmosphere and winds

<code>atmosphere/T-R, T-sl-R, delta-T</code>	
<code>atmosphere/P-psf, P-sl-psf</code>	
<code>atmosphere/rho-slugs_ft3</code>	
<code>atmosphere/a-fps</code>	speed of sound
<code>atmosphere/density-altitude</code>	
<code>atmosphere/pressure-altitude</code>	
<code>atmosphere/wind-{north,east,down}-fps</code>	steady wind in NED
<code>atmosphere/total-wind-{north,east,down}-fps</code>	wind+gust+turb
<code>atmosphere/turb-type</code>	turbulence model (0..4)
<code>atmosphere/turb-rate-rad_sec</code>	current angular turbulence

20.3 Position

<code>position/lat-geod-deg, lat-gc-rad</code>	geodetic and geocentric lat
<code>position/long-gc-deg</code>	
<code>position/h-sl-ft, h-sl-meters</code>	altitude above MSL
<code>position/h-agl-ft</code>	altitude above ground
<code>position/geod-alt-ft</code>	
<code>position/distance-from-start-mag-mt</code>	great-circle distance
<code>position/terrain-elevation-asl-ft</code>	

20.4 Attitude

<code>attitude/phi-rad, theta-rad, psi-rad</code>	Euler
<code>attitude/phi-deg, theta-deg, psi-deg</code>	degrees
<code>attitude/heading-true-rad</code>	
<code>attitude/pitch-rad, roll-rad</code>	
<code>/sim/attitude/q[0..3]</code>	quaternion components (if exposed)

20.5 Velocities

<code>velocities/vt-fps</code>	true airspeed
<code>velocities/vc-kts</code>	calibrated airspeed
<code>velocities/ve-kts</code>	equivalent airspeed
<code>velocities/vg-fps</code>	ground speed
<code>velocities/mach</code>	
<code>velocities/u-fps, v-fps, w-fps</code>	body-frame velocity
<code>velocities/u-aero-fps</code>	body-frame airmass-relative
<code>velocities/p-rad_sec, q-rad_sec, r-rad_sec</code>	body angular rates
<code>velocities/p-aero-rad_sec</code>	aero-frame angular rates
<code>velocities/vdown-fps</code>	NED down velocity (=climb rate)

20.6 Aerodynamic state

aero/alpha-rad, alpha-deg	
aero/beta-rad, beta-deg	
aero/alphadot-rad_sec, betadot-rad_sec	
aero/mag-alpha-rad, mag-beta-rad	alpha , beta
aero/qbar-psf, qbarUW-psf, qbarUV-psf	
aero/h_b-mac-ft	h/c for ground effect
aero/bi2vel, ci2vel	
aero/cl-squared	CL ² from previous tick
aero/stall-hyst-norm	stall hysteresis switch (0 or 1)

20.7 Forces and moments (body frame)

forces/fbx-aero, fby-aero, fbz-aero	aerodynamic
forces/fbx-prop, fby-prop, fbz-prop	propulsion
forces/fbx-gear, fby-gear, fbz-gear	ground reactions
forces/fbx-external, fby-external, fbz-external	
forces/fbx-buoyant, fby-buoyant, fbz-buoyant	
forces/fbx-total, fby-total, fbz-total	sum (used by Accelerations)
moments/l-aero, m-aero, n-aero	
moments/l-prop, m-prop, n-prop	
moments/l-gear, m-gear, n-gear	
moments/l-total, m-total, n-total	

20.8 Accelerations

accelerations/udot-ft_sec2, vdot-ft_sec2, wdot-ft_sec2	
accelerations/pdot-rad_sec2, qdot-rad_sec2, rdot-rad_sec2	
accelerations/a-pilot-x-ft_sec2	at the EYEPOINT
accelerations/n-pilot-x-norm	load factor at EYEPOINT, g
accelerations/Nz	normal load factor

20.9 FCS commands and positions

fcs/elevator-cmd-norm	-1..+1 from pilot/script
fcs/aileron-cmd-norm	
fcs/rudder-cmd-norm	
fcs/pitch-trim-cmd-norm	
fcs/roll-trim-cmd-norm	
fcs/yaw-trim-cmd-norm	
fcs/flap-cmd-norm	
fcs/throttle-cmd-norm[n]	
fcs/mixture-cmd-norm[n]	
fcs/elevator-pos-rad, fcs/elevator-pos-deg, fcs/elevator-pos-norm	
fcs/left-aileron-pos-rad, ...	
fcs/rudder-pos-rad, ...	
fcs/flap-pos-deg, fcs/flap-pos-norm	
fcs/speedbrake-pos-rad	
fcs/left-brake-cmd-norm, right-brake, center-brake	
fcs/steer-cmd-norm	

20.10 Propulsion

propulsion/engine[n]/thrust-lbs	
propulsion/engine[n]/fuel-flow-rate-pps	per-second
propulsion/engine[n]/fuel-flow-rate-gph	
propulsion/engine[n]/n1, n2	turbine spool speeds
propulsion/engine[n]/rpm	prop or piston rpm
propulsion/engine[n]/torque-lbsft	
propulsion/engine[n]/power-hp	piston
propulsion/engine[n]/map-inhg	manifold pressure
propulsion/engine[n]/egt-degf	exhaust temp
propulsion/engine[n]/cht-degf	cylinder head
propulsion/engine[n]/oil-pressure-psi	
propulsion/engine[n]/oil-temp-degf	
propulsion/tank[n]/contents-lbs, capacity-lbs	
propulsion/total-fuel-lbs	

20.11 Simulation control

simulation/do_simple_trim	1 = longitudinal trim, 2 = ground, ...
simulation/do_trim	full trim mode
simulation/reset	reset to initial conditions
simulation/terminate	stop the script
simulation/pause	freeze the model
simulation/gravity-model	0 spherical, 1 WGS-84
simulation/gravitational-torque	spacecraft tidal torque on/off
simulation/notify-time-trigger	
simulation/randomseed	

Chapter 21: Glossary and Symbol Index

Symbol	Meaning
α (alpha)	Angle of attack — angle between the body X axis and the projection of relative wind onto the body X-Z plane.
β (beta)	Sideslip angle — angle between relative wind and the body X-Z plane.
$\dot{\alpha}$ (alphadot)	Time rate of change of α .
$\bar{q}$ (qbar)	Dynamic pressure, $\frac{1}{2}\rho V^2$.
ρ (rho)	Atmospheric density.
V_t	True airspeed (magnitude of velocity relative to the air mass).
V_c	Calibrated airspeed (compressibility-corrected indicated airspeed).
V_e	Equivalent airspeed (density-corrected indicated airspeed).
M	Mach number, V_t/a .
a	Speed of sound.
p, q, r	Body-frame roll, pitch, yaw angular rates.
u, v, w	Body-frame translational velocity components.
ϕ, θ, ψ	Bank, pitch, heading Euler angles (3-2-1 sequence).
γ (gamma)	Flight path angle (positive climb).
S	Wing reference area.
b	Wing span.
$\bar{c}$ (cbar)	Mean aerodynamic chord.
AR	Aspect ratio, b^2/S .
e	Oswald efficiency factor (0.7-0.95 typical).
AERORP	Aerodynamic reference point — origin for aerodynamic moments.
VRP	Visual reference point — origin used by external viewers.
EYEPOINT	Pilot eye position.
AGL	Above ground level.
MSL	Mean sea level.
WGS-84	World Geodetic System 1984 — Earth ellipsoid model used by GPS and JSBSim.
ECEF	Earth-centred, Earth-fixed (rotates with the planet).
ECI	Earth-centred inertial (non-rotating).
NED	North-East-Down local tangent plane.
LCP	Linear complementarity problem — formulation of contact friction.
FDM	Flight Dynamics Model.
FCS	Flight Control System.
IC	Initial conditions.
BSFC	Brake specific fuel consumption.
MAP	Manifold absolute pressure (piston engines).
N1, N2	Low-pressure and high-pressure spool speeds (turbine engines).
TSFC	Thrust specific fuel consumption (turbines).
CFD	Computational Fluid Dynamics.
RANS	Reynolds-Averaged Navier-Stokes.
URANS	Unsteady RANS.
LES	Large-eddy simulation.

AVL	Athena Vortex Lattice (Drela).
DATCOM	USAF Stability and Control DATCOM handbook.
POH	Pilot's Operating Handbook.
TAS	True airspeed ($=V_t$).
IAS	Indicated airspeed.
CAS	Calibrated airspeed ($=V_c$).
EAS	Equivalent airspeed ($=V_e$).
KCAS / KTAS / KIAS	Knots calibrated/true/indicated airspeed.

End of the Ultimate JSBSim Reference. Bug reports and improvements welcome at <https://github.com/JSBSim-Team/jsbsim>.